DAY 1 · THURSDAY 22 OCTOBER



Large Language Models I

Language modeling, from n-grams to word embeddings and attention

Pavlos Protopapas
Harvard University

Outline

- Natural Language Processing
- Text Preprocessing
- **Language Modeling**
  - **Unigrams**
  - **Bigrams**

Natural Language Processing

Natural Language Processing (NLP) is the field of study that focuses on the interaction between computers and humans through natural language, with the goal of enabling machines to understand, interpret, and respond to human language in meaningful ways.

Natural Language Processing | Applications

Text recognition:

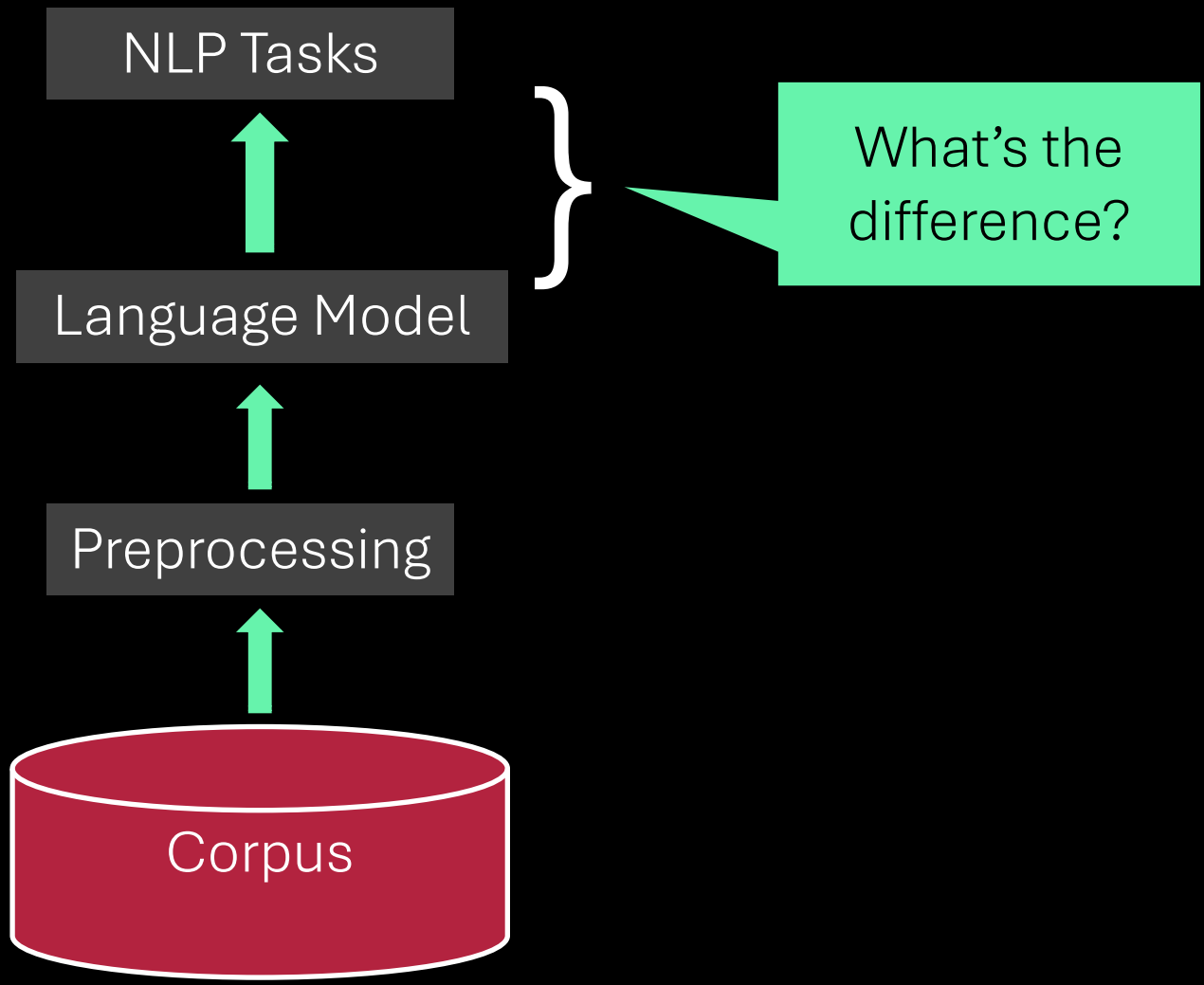


Natural Language Processing | Applications

Sentence prediction:



Pipeline


NLP Tasks
Language Model
Preprocessing
Corpus
What's the difference?

Game Time

How do we teach a toddler to answer a question? Choose the correct order.

I. **Teach the toddler to answer questions**

II. Teach the toddler language

A. I -> II

B. II -> I

C. I don’t want the toddler to answer questions

D. None of the above

NLP Task vs. Language Model

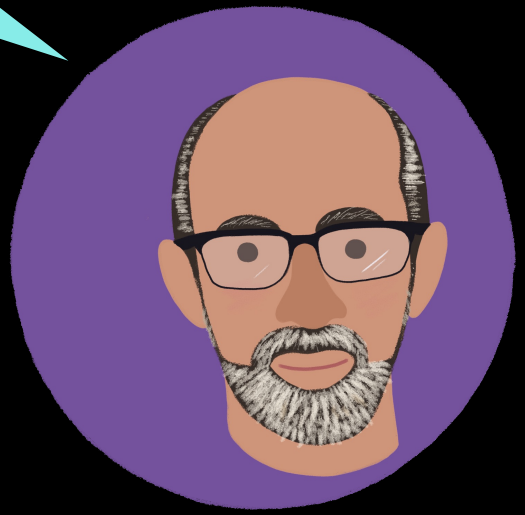
Similarly, we would want to teach our model to understand language before any downstream tasks like NER, question answering, etc.



Then what can we do about it?

We can tap into the vast ocean of unstructured text on the internet.





NLP Task vs. Language Model

3. We train the language model using the data that we got from the internet.



NLP Task vs. Language Model

3. We train the language model using the data that we got from the internet.

is the Scientific Program Director <EOS>

Language Model



Input: Pavlos is the Scientific Program Director

NLP Task vs. Language Model

4. Finally, we fine-tune our pre-trained language model to perform an NLP task, such as part-of-speech tagging.

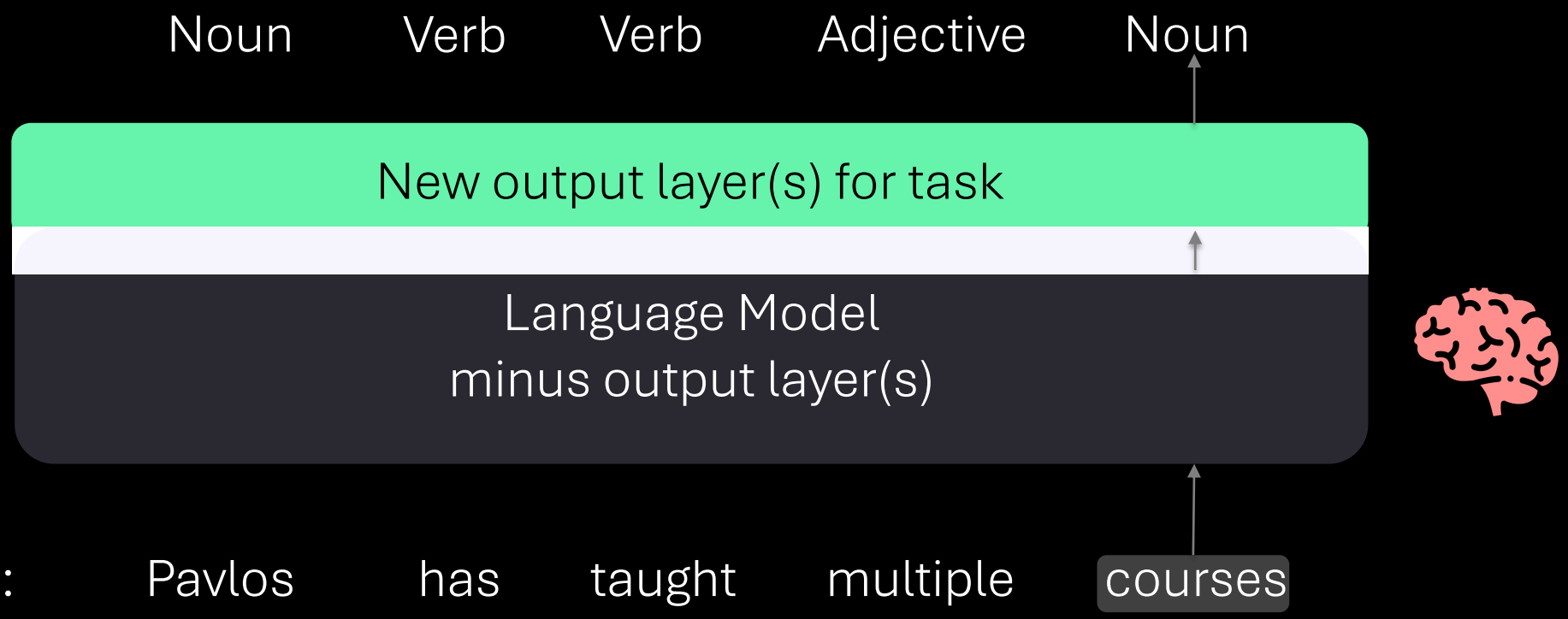
Replace the LM's output layer with a different one for our new task!



Input: Pavlos has taught multiple courses

NLP Task vs. Language Model

4. Finally, we fine-tune our pre-trained language model to perform an NLP task, such as part-of-speech tagging.



Language Modeling | Starting Point

Text is comprised of a sequence of words that convey meaning. To process it, we break the text down into smaller units known as **tokens**.

Thus, a **sentence** is transformed into a **sequence** of **tokens**.

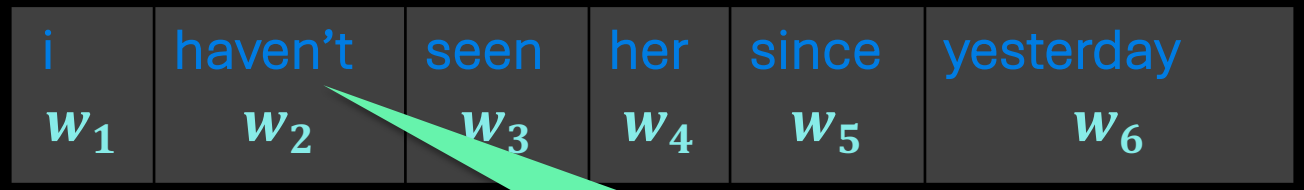
All the unique tokens from a dataset make up the **vocabulary**.

The larger the vocabulary, the larger the training set required.

We'll also see reasons to include additional 'special' tokens in our vocabulary.

Tokenization





Hyphenated phrases like "award-winning" are not split.

Usually, punctuation is removed before this stage and capitalization is set to lower case.

Conjunctions such as haven't are not split.

Tokenization

Second Approach: sub-word tokenization

Split the words on **statistically significant** fragments.

Tokens often lose their direct interpretabil**ity, b**ut the approach is more flexible.

Tokenization

Second Approach: sub-word tokenization

Split the words on **statistically significant** fragments.

How can we define a token?

Sub-word tokenization removes the **full word**, but keeps the smaller pieces that appear frequently across many words.

the	award	-	winning	actor	arrived	today
w_1	w_2	w_3	w_4	w_5	w_6	w_7

turn	the	handle	counter	clock	wise
w_1	w_2	w_3	w_4	w_5	w_6

Tokenization is an area of research in itself.

Language Modeling | Formal Definition

We model any sequential data as follows:

$$P(x_1, \ldots, x_T) = \prod_{t=2}^{T} p(x_t | x_{t-1}, \ldots, x_1)$$

We multiply a new term for every token — the joint probability is their product.

Joint distribution of all measurements

Conditional probability of an event, depends on all events that occurred before it

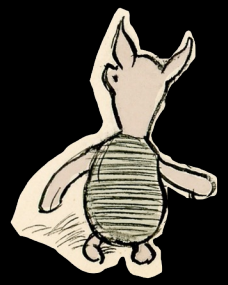
Example

The probability of the the next on-screen character can be computed as:

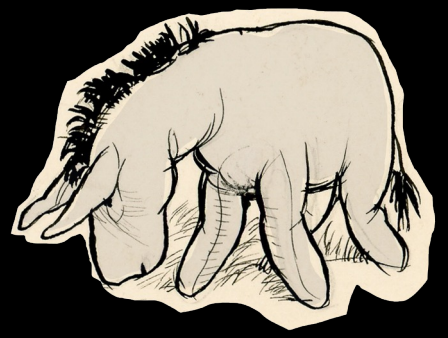
Scene 1



Scene 2



Scene 3



$$P(\text{Scene 1}, \text{Scene 2}, \text{Scene 3}) = \underbrace{P(\text{Scene 1})}_{\text{Scene 1}}\ \underbrace{P(\text{Scene 2} \mid \text{Scene 1})}_{\text{Scene 2}}\ \underbrace{P(\text{Scene 3} \mid \text{Scene 2}, \text{Scene 1})}_{\text{Scene 3}}$$

Example

Having trained a **language model** means that we can reasonably predict the behavior of the sequences.

If we have a sequence of length N, we can predict the most likely next event (i.e., sequence of length $N + 1$)

$$P(\text{  }, \text{  }, S_3) = P(\text{  }) \; P(\text{  } \mid \text{  }) \; P(S_3 \mid \text{  }, \text{  })$$

Scene 1 Scene 2 Scene 3

Language Modeling as a Sequence of Events

A **language model** estimates the probability of any sequence of words.

Let $\boldsymbol{X}$ = "**Shiv was late for class**"

w_1 w_2 w_3 w_4 w_5

$P(\boldsymbol{X}) = P(\text{"}\textbf{Shiv was late for class}\text{"})$

Language Modeling | Unigrams

How can we build a language model?

$$P(x_1, \dots, x_T) = \prod_{t=1}^{T} p(x_t)$$

Naive Approach: Unigram model

Assume each word is independent of all others.

Count how often each word occurs (in the training data).

Let $\boldsymbol{X}$ = "**Pavlos loves giving surprise quizzes**"

w_1 w_2 w_3 w_4 w_5



Language Modeling | Unigrams

How can we build a language model?

Naive Approach: Unigram model

Assume each word is independent of all others

Let $\boldsymbol{X}$ = "**Pavlos loves giving surprise quizzes**"

w_1 w_2 w_3 w_4 w_5

$\mathrm{P}(\boldsymbol{X}) = P(\textbf{Pavlos})\, P(\textbf{loves})\, P(\textbf{giving})\, P(\textbf{surprise})\, P(\textbf{quizzes})$

= 0.00015 * 0.01 * 0.004 * 0.03 * 0.0035

= 6.3×10^{-13}

You calculate and store each of these probabilities from the training corpus.

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t)$$

Language Modeling | Unigram Issues

Context does not affect probabilities

$P(\text{“Pavlos loves giving surprise quizzes”}) = P(\text{“quizzes loves giving surprise Pavlos”})$

Sequence generation: What's the most likely next word?

Pavlos loves giving surprise quizzes all ___

Pavlos loves giving surprise quizzes all the

Pavlos loves giving surprise quizzes all the the

Out of vocabulary words

$$P(\text{“hungry”}) = 0$$

Language Modeling | Unigram Issues

Before Smoothing :

$$P(\text{“hungry”}) = 0$$

After Smoothing :

$$P(\text{“UNK”}) = \frac{\alpha}{|W| + \alpha\,|V|} > 0$$

Smoothing allows probability of “UNK” token to be non-zero, enabling the model to predict out-of-vocabulary words.

Language Modeling | Bigrams

How can we build a language model that uses context?

Easiest Approach: bigram model

Look at **pairs** of consecutive words

$$P(x_1, \dots, x_T) = p(x_1) \prod_{t=2}^{T} p(x_t | x_{t-1})$$

probability

Let $\boldsymbol{X}$ = "Pavlos loves giving surprise quizzes"

w_1 w_2 w_3 w_4 w_5

$\mathrm{P}(\boldsymbol{X}) = P(\text{Pavlos})\, P(\text{loves}|\text{Pavlos})\, P(\text{giving}|\text{loves})\, P(\text{surprise}|\text{giving}) \times P(\text{quizzes}|\text{surprise})$

Language Modeling | Bigrams

You calculate each of these probabilities by simply counting the occurrences:

$$P(quizzes|surprise) = \frac{C(W_1W_2)}{C(W_1w)} = \frac{C(W_1W_2)}{C(W_1)} = \frac{Total\ number\ of\ \text{“}surprise\ quizzes\text{”}}{Total\ number\ \text{“}surprise\text{”}}$$

We must store all these probabilities somewhere to be accessed at inference (prediction) time.

Let $\boldsymbol{X}$ = “Pavlos loves giving surprise quizzes”

probability

w_1 w_2 w_3 w_4 w_5

$\mathrm{P}(\boldsymbol{X})$ = P(Pavlos) P(loves|Pavlos) P(giving|loves) P(surprise|giving) P(quizzes|surprise)

Language Modeling | Bigrams

- When a word is **out-of-vocabulary**, it's assigned a probability of 0, causing the entire sentence or sequence to also have **a probability of 0**.
- **More context** (like trigrams, 4-grams) is often **desired**, but sparsity is a challenge due to **infrequent** subsequences.
- As we expand the window size for context, **storage issues** arise.
- **Raw counts** fail to convey deep **semantic** relationships, such as the similarity between 'vehicle' and 'car'.

HarvardX LLMs Course

Language models with neural networks

Pavlos Protopapas

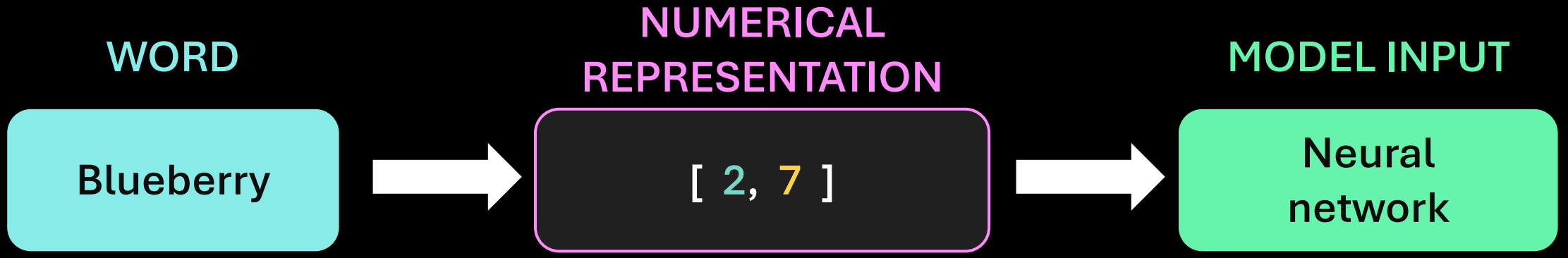


Outline

- **Why Do We Need Neural Networks?**
- Word Embeddings: Representing Words as Vectors
- A Feed-Forward Neural Language Model
- Training the Network: Loss and Backpropagation
- Where We Go From Here

Neural Networks Operate on Numbers

Before a model can use a word, we must represent it numerically.



an ordered list of numbers
— as long as we choose

We will begin with deliberately interpretable toy values.

A Fruit Can Be Represented by Two Numbers

Toy vector components: **Size** · **Sweetness**

Fruit	Size	Sweetness
Blueberry	2	7
Kiwi	5	5
Lemon	6	2
Banana	7	6

What is a vector?

Just an ordered list of numbers.

1 · Size

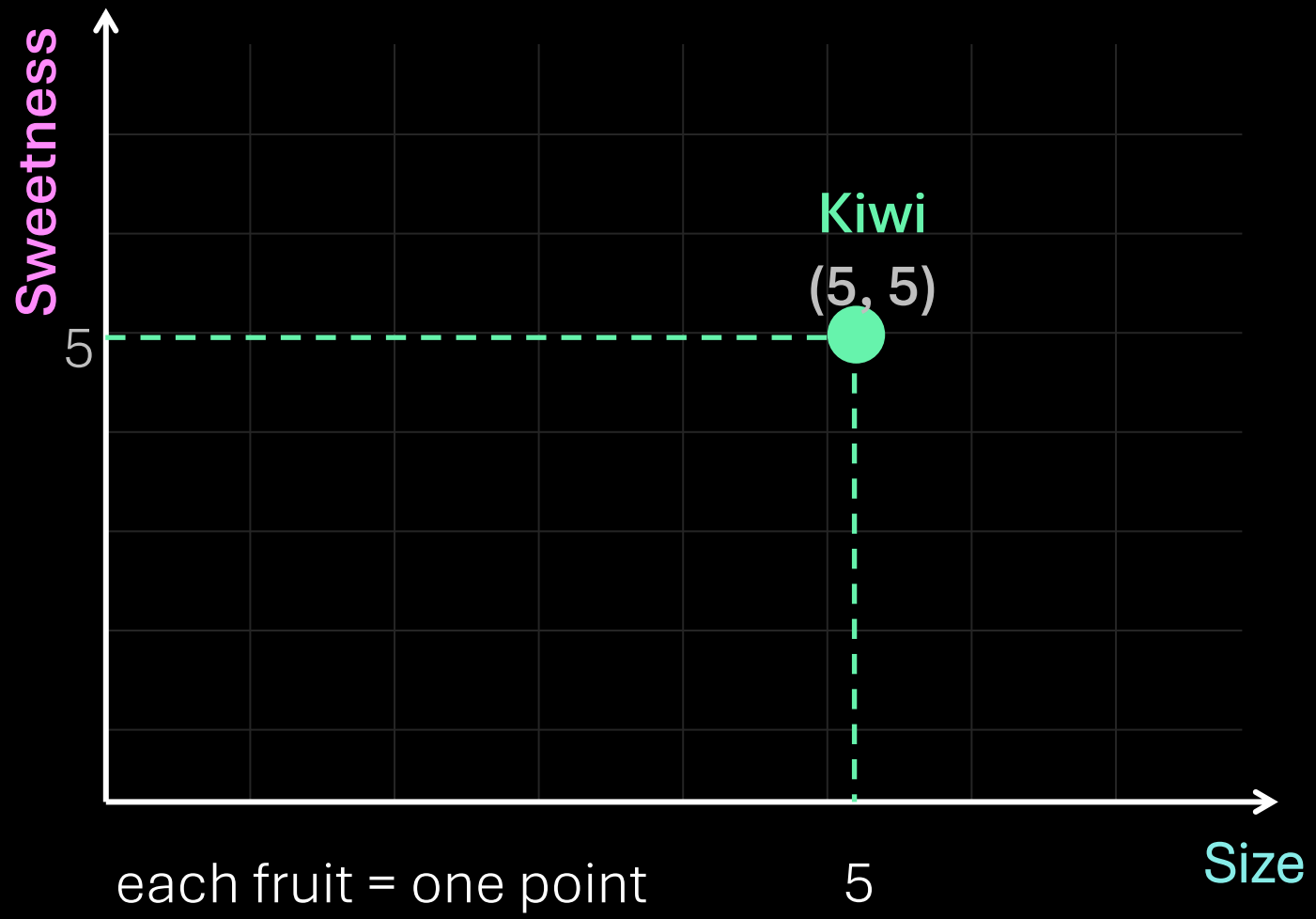
2 · Sweetness

The slots never move, so two fruits can be compared number by number.

each row is one vector **Blueberry** → [2, 7] 3 numbers = 3 dimensions

Illustrative teaching values — deliberately simplified, not nutritional measurements.

A Vector is Just an Address in This Space

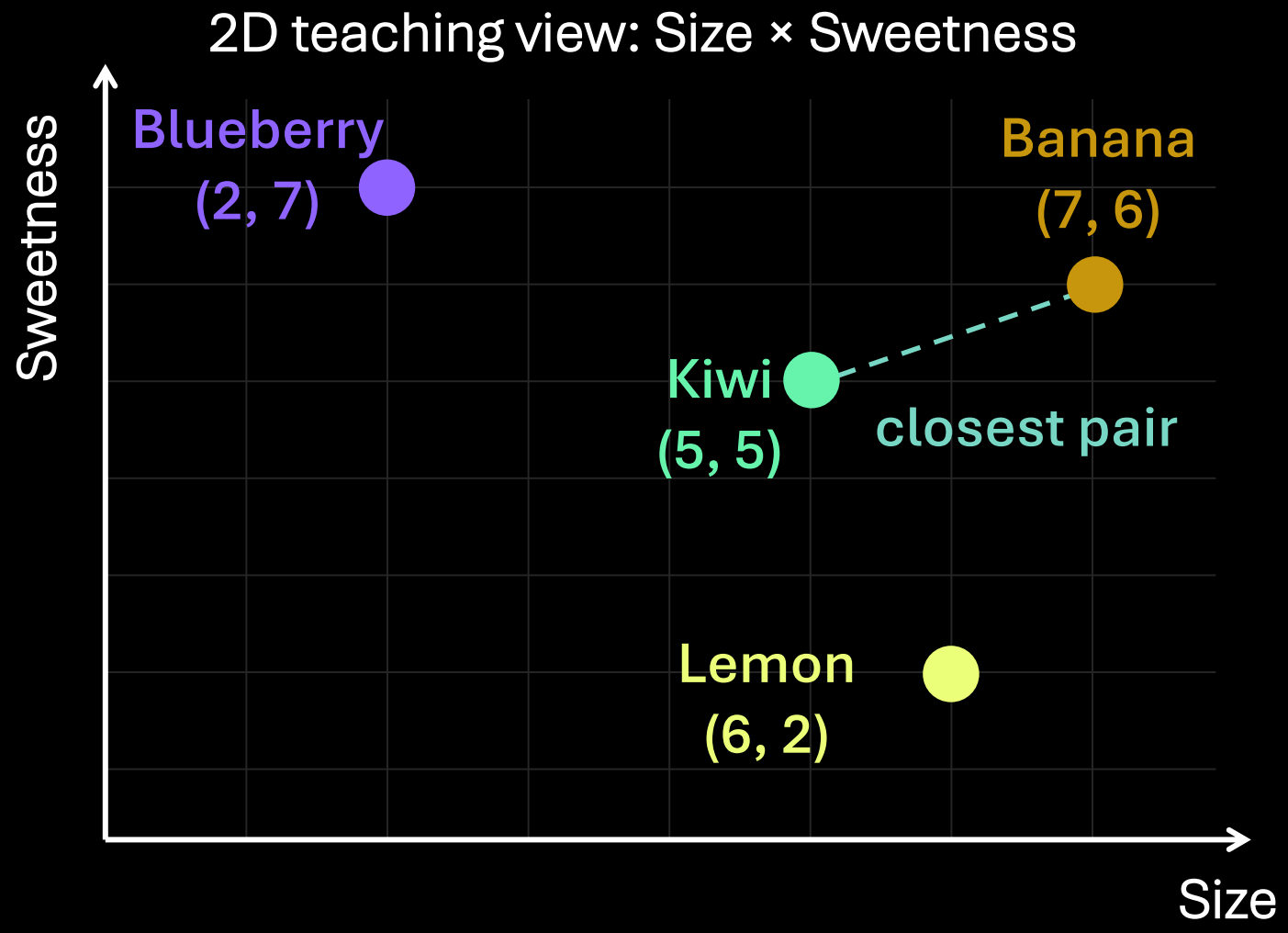


Kiwi = [5, 5]

- **First number** → how far along **Size**
- **Second number** → how far up **Sweetness**
- The order is fixed, so the two numbers always point to the same spot.

More numbers = more dimensions. Same idea — just harder to draw.

Nearby Points Represent More Similar Fruits



Nearby points share more of the selected characteristics.

Distance gives us a simple notion of similarity.

Two components mean we can draw every fruit as a point on this plane.

Illustrative teaching values—not nutritional measurements.

Word Embeddings Keep the Geometry, Not the Labels

Toy fruit vector

chosen and named by us

Size	2
Sweetness	7



Real word embedding

learned from language data

d_1	0.12
d_2	−0.47
d_3	0.08
⋮	⋮
d_{200}	0.31

The learned dimensions are usually not human-interpretable.

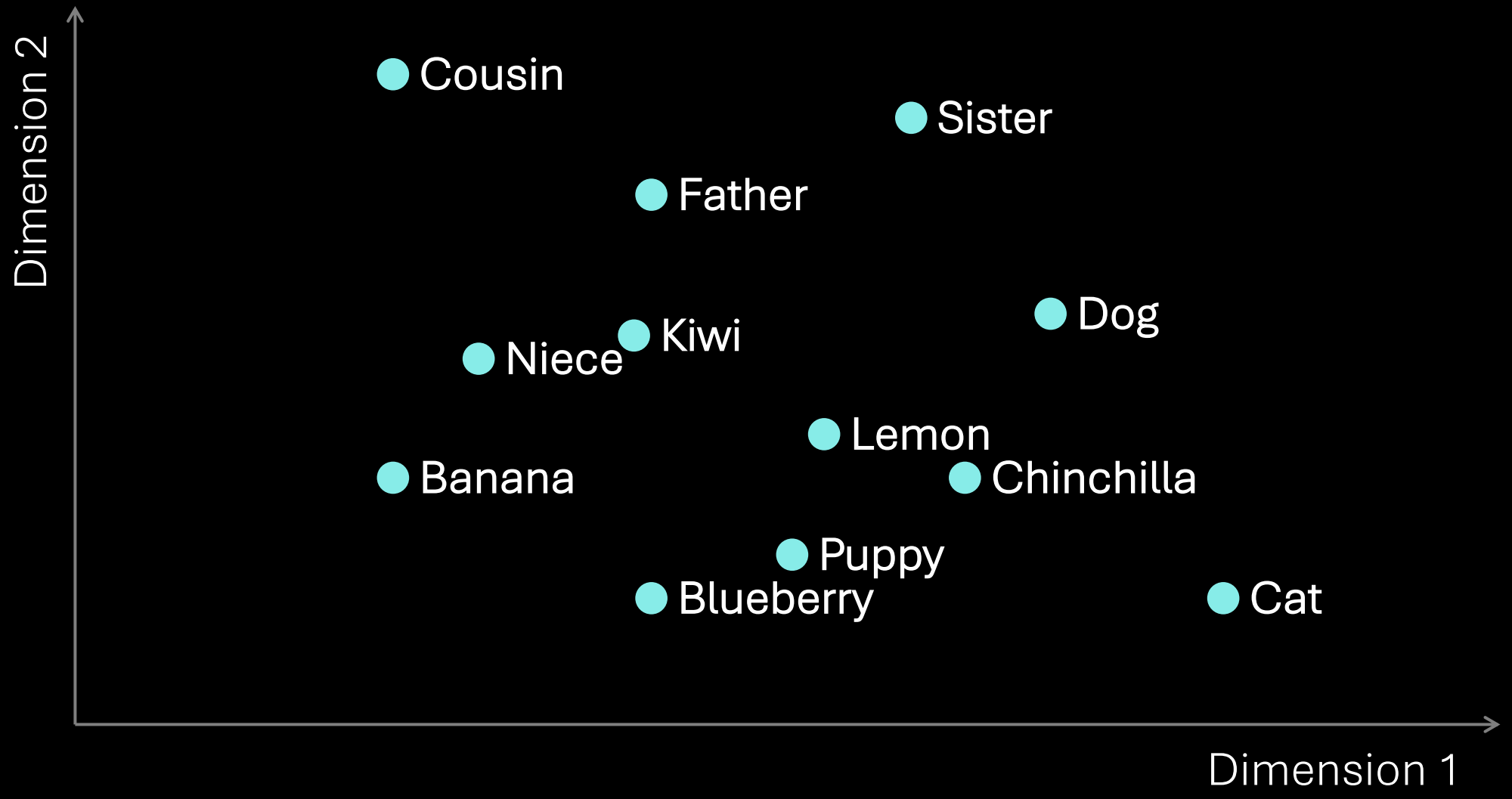
Word Embeddings Start Out Random

Twelve words, placed at random before any training.

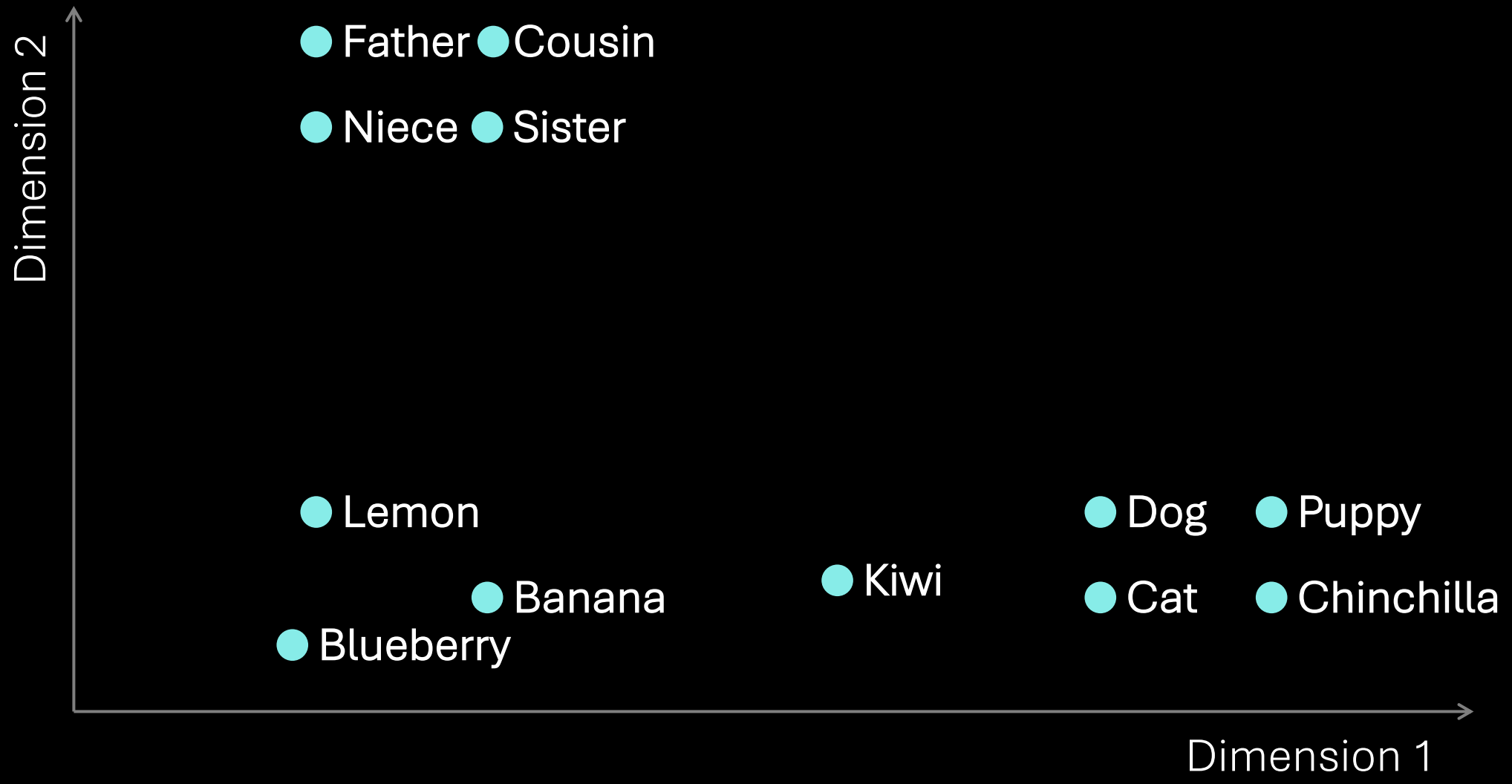


Training Pulls Related Words Together

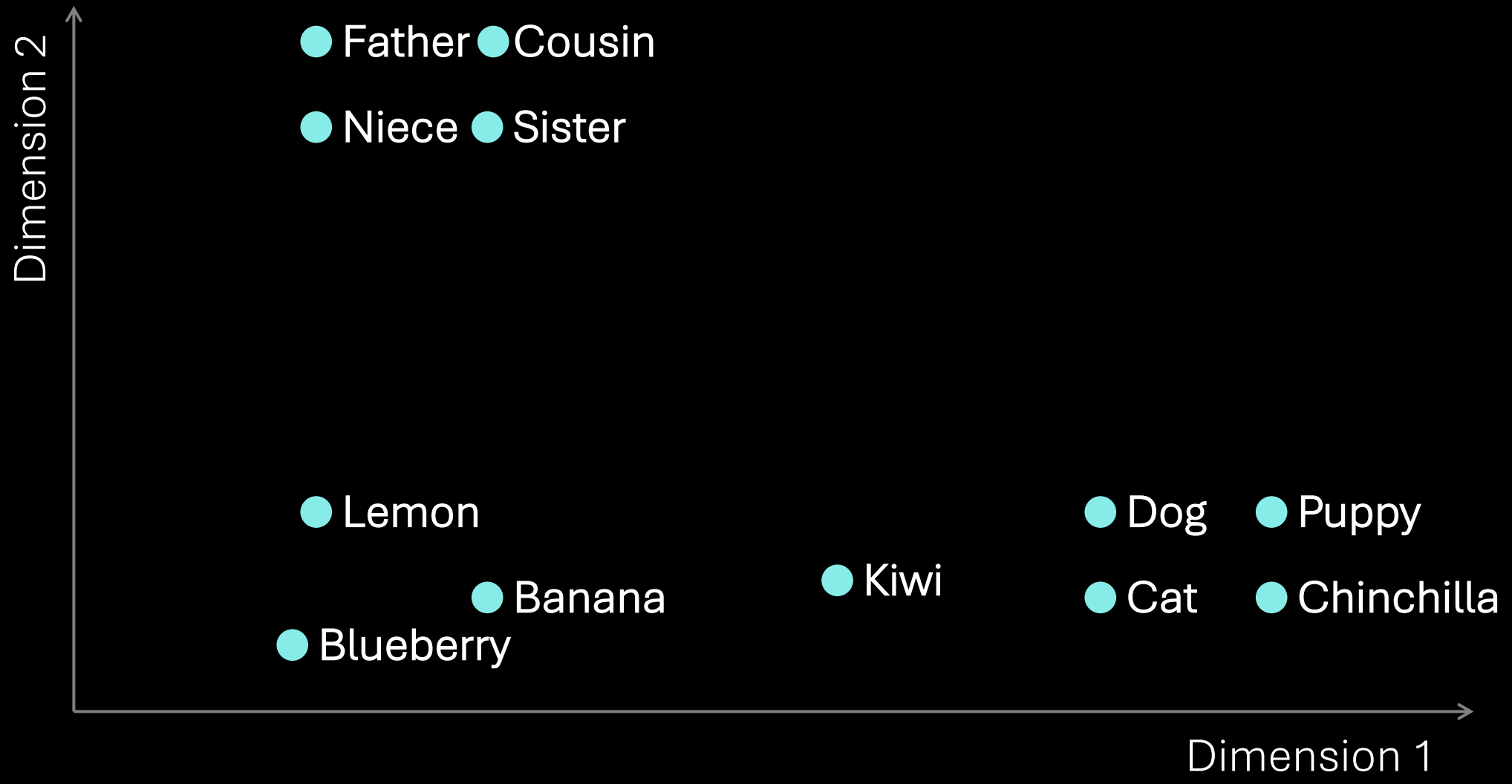
Training nudges related words closer together.



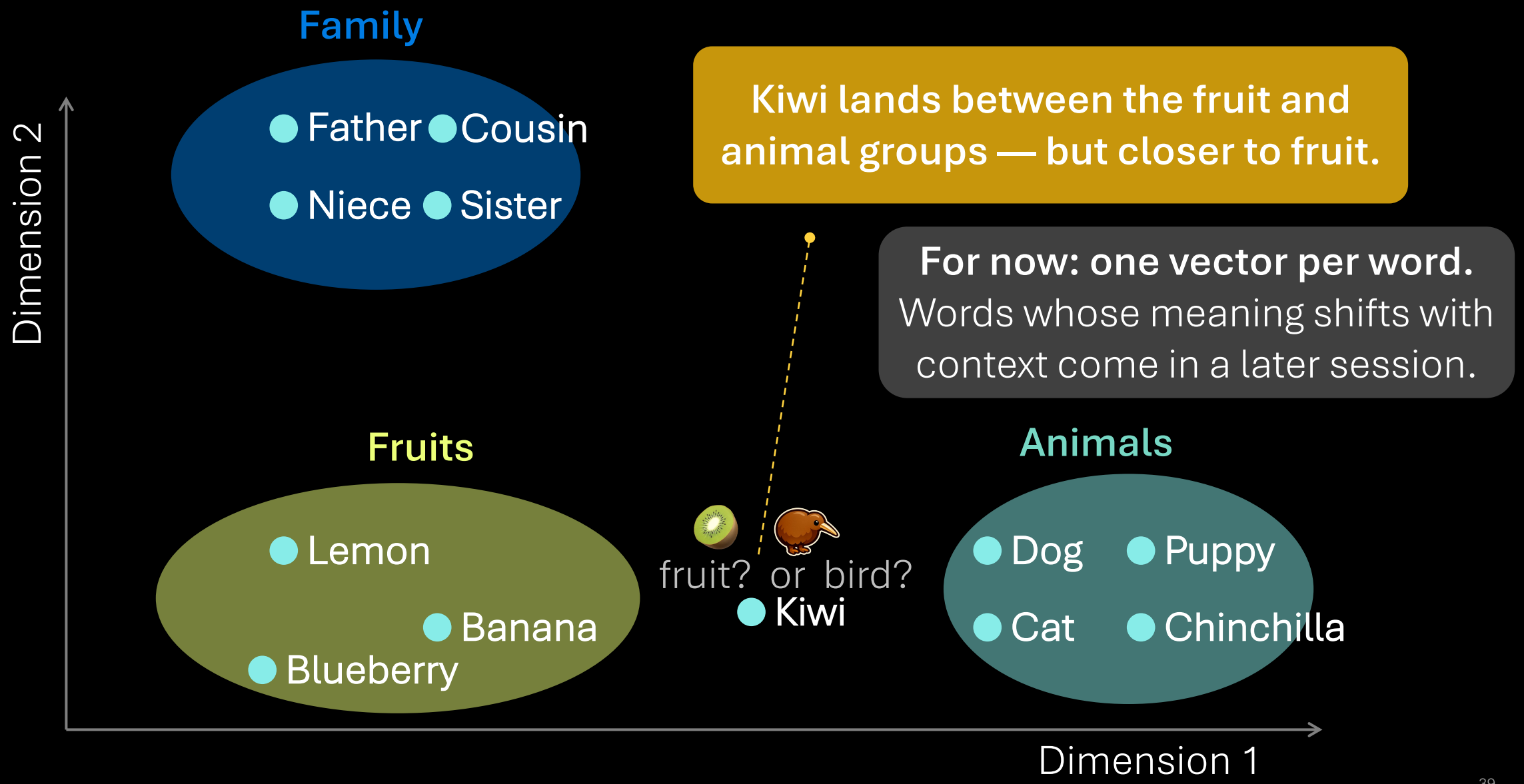
Learned Geometry Becomes Meaningful



Learned Geometry Becomes Meaningful



Clusters Form — and the Outliers are Informative



Learned Numbers | The Raw Values

Each cell is one learned floating-point scalar.

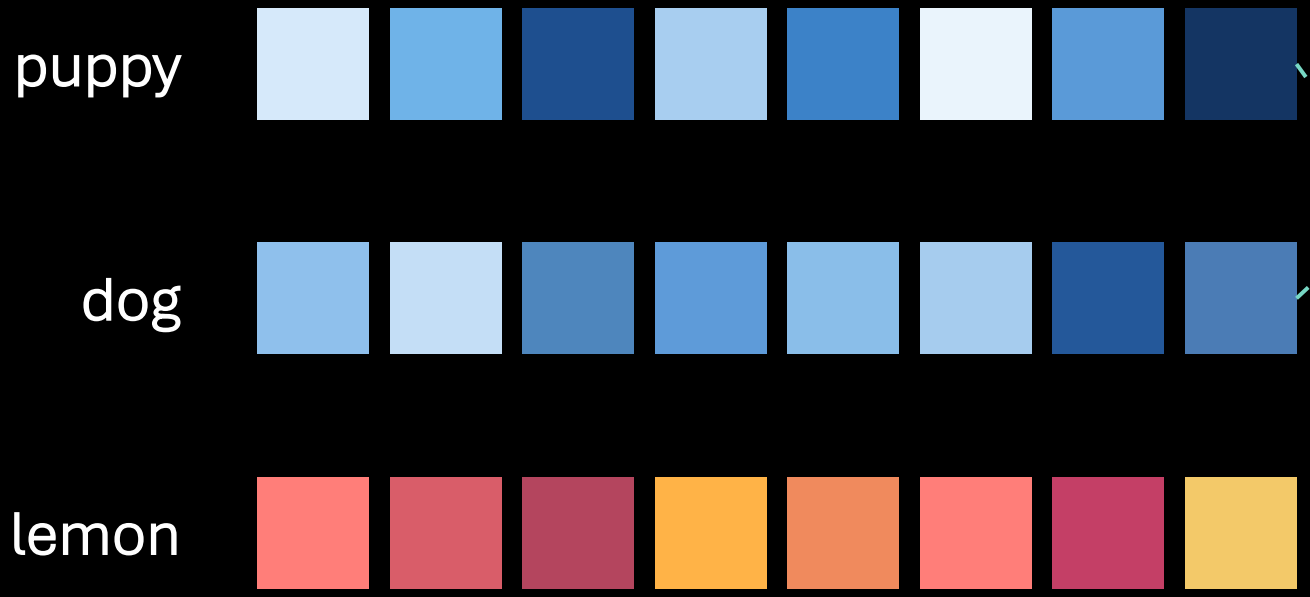
puppy	0.10	0.37	0.77	0.23	0.56	0.05	0.46	0.89
dog	0.29	0.15	0.55	0.45	0.31	0.24	0.73	0.58
lemon	−0.40	−0.53	−0.63	−0.39	−0.44	−0.40	−0.61	−0.32

Which two rows are similar? Hard to tell from raw numbers...



Learned Numbers Create Useful Geometry

Each cell is one learned floating-point scalar.

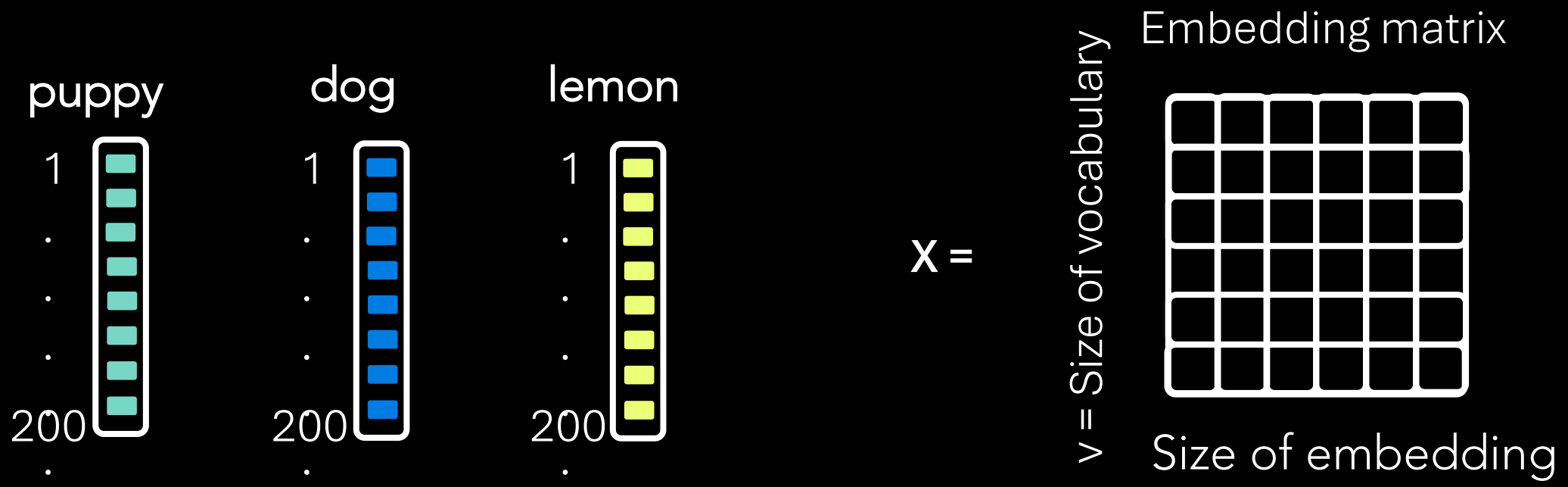


Similar words can land close together.

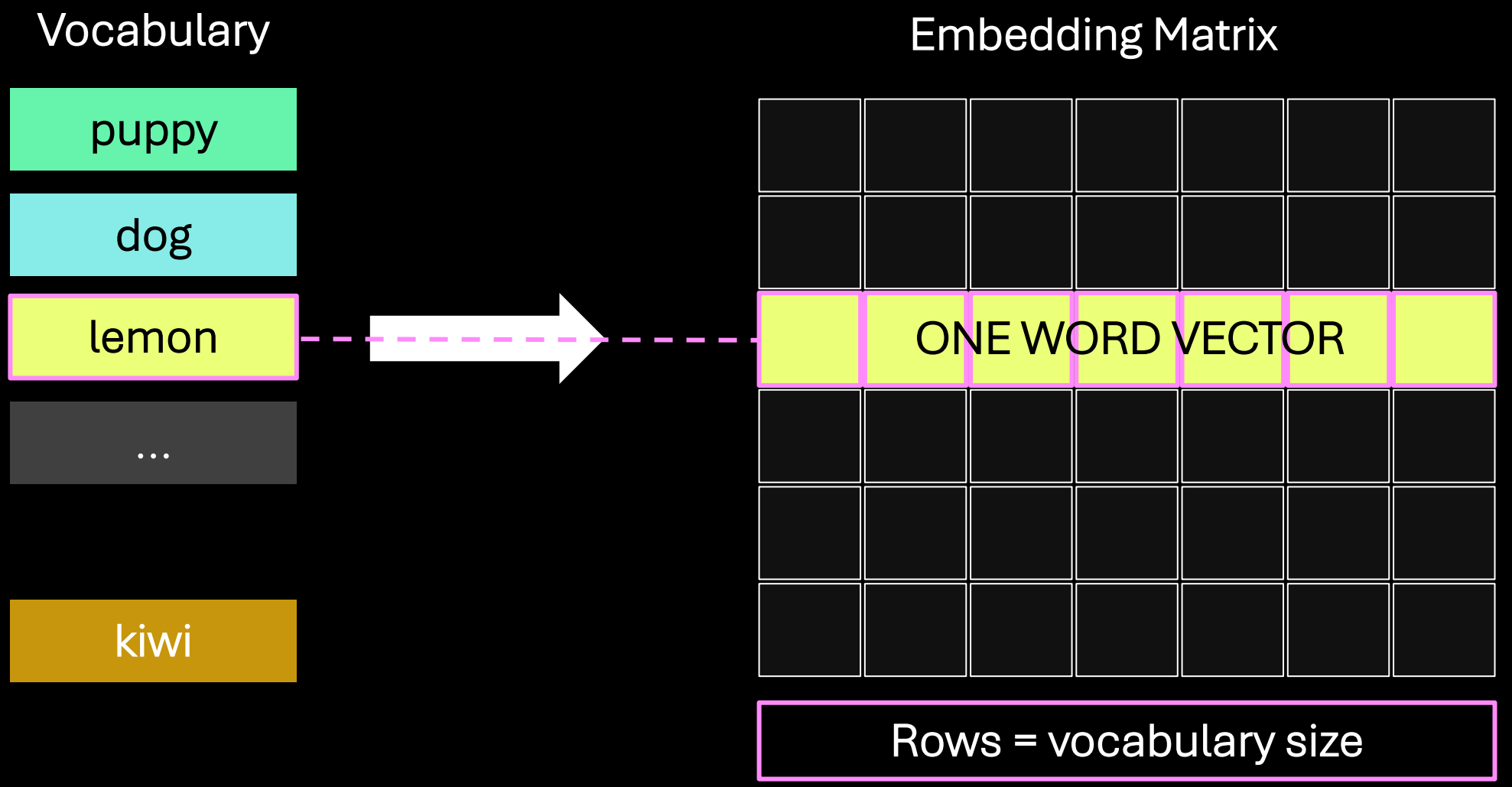
The individual dimensions are generally not interpretable.

Embedding Matrix | One Vector per Word

The embedding matrix stores one vector for every word in the vocabulary.



Embedding Matrix | One Row per Word



Embedding Matrix | One Column per Dimension

Vocabulary

puppy

dog

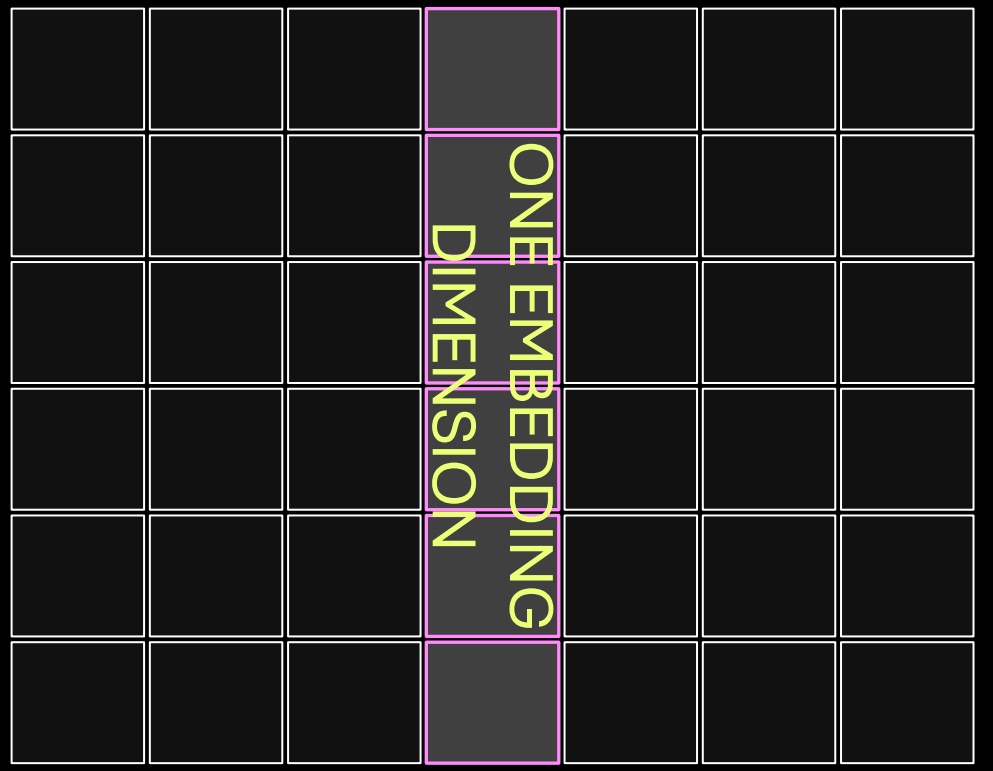
lemon

…

kiwi

the same component (the nth) of every embedding

Embedding matrix



Columns = embedding dimension d

d is a hyperparameter we choose, e.g., 32, 64, 128 (768+ in modern LLMs)

In Practice, the Embedding Matrix Already Exists

Before we build a language model, we usually start from an embedding matrix that has already been trained.

Word2Vec Mikolov et al., 2013

Learns its vectors by predicting nearby words from context, across a huge collection of text.

GloVe Pennington et al., 2014

Learns its vectors from how often words co-occur with each other across the whole corpus.

Either way, the result is the same: one ready-made vector per word, the embedding matrix we plug into our model.

See the supplementary materials for more on Word2Vec and GloVe.

From Embeddings to a Language Model

How can we use these embeddings to build a Language Model?

Remember, we only need a system that can estimate:

$$P(x_{t+1}|x_t, x_{t-1}, \ldots, x_1)$$

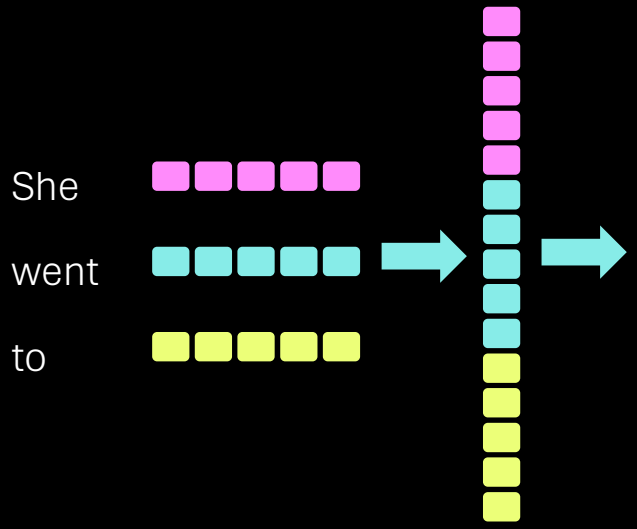
next word | previous words

Example input sentence



Inside the Box | Concatenate, Hidden Layers, Softmax

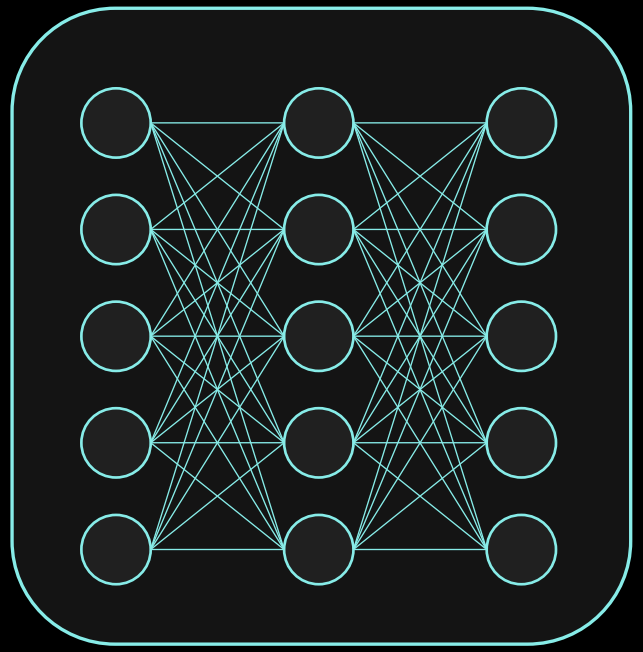
1. Concatenate the window



One long input vector: 3 words × 200 dims = 600 numbers

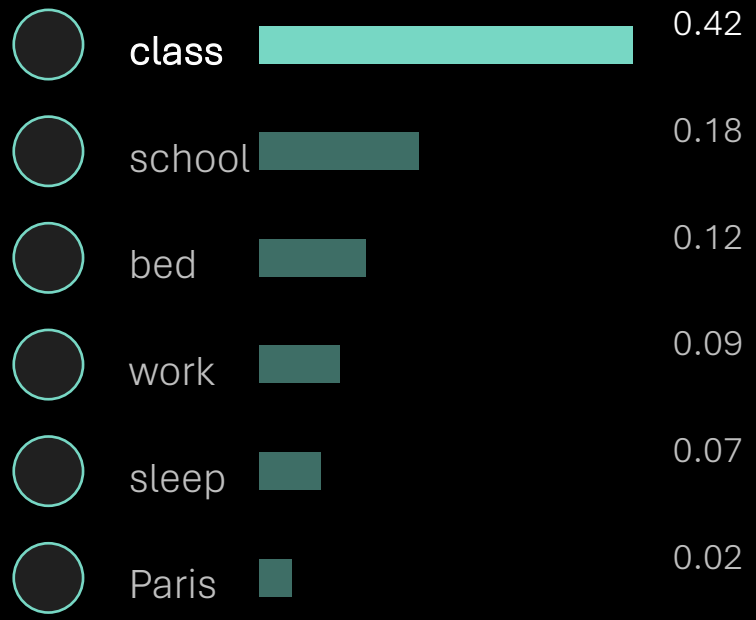
Each color block is one word's embedding — concatenation is the only "combining" here.

2. Hidden layers



hidden layers — fully connected, learning feature combinations

3. Output + softmax



softmax
scores → probabilities that sum to 1

Review | What the Hidden Layer and Softmax Do

Hidden layer

$$h = f(W_1 x + b_1)$$

linear map, then an elementwise nonlinear activation

- The linear part mixes every input dimension — each unit weights all 600 numbers.
- The nonlinearity f (tanh or ReLU) is what lets the model represent non-linear structure.
- h is a learned feature vector, not a probability — its entries can be any real number.

Softmax

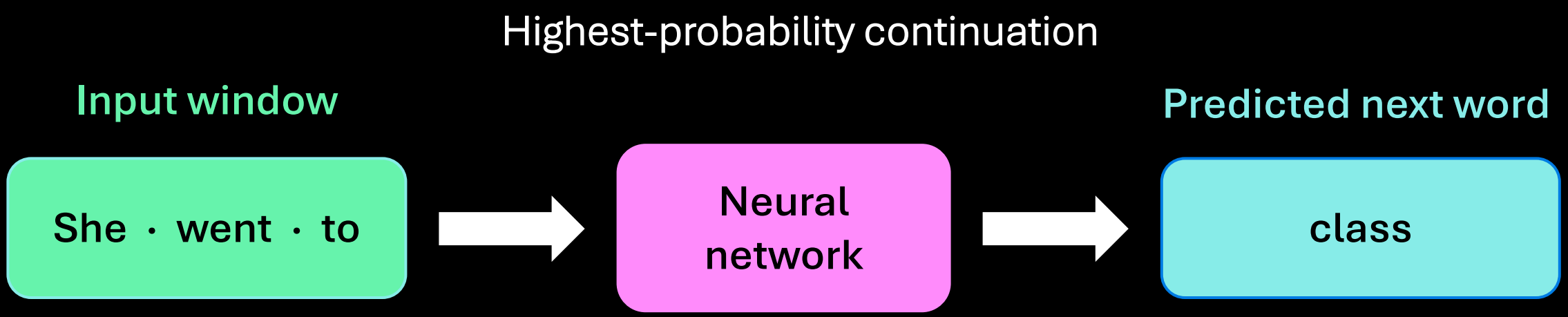
$$p_i = \frac{exp(z_i)}{\sum_j exp(z_j)}$$

$$z = W_2 h + b_2$$

- Takes one real score per vocabulary word and returns a probability for each.
- Exponentiating keeps every value positive; dividing by the sum forces them to add to 1.
- Order is preserved: the largest score always gets the largest probability.

Both are standard building blocks — this slide is a refresher before we use them.

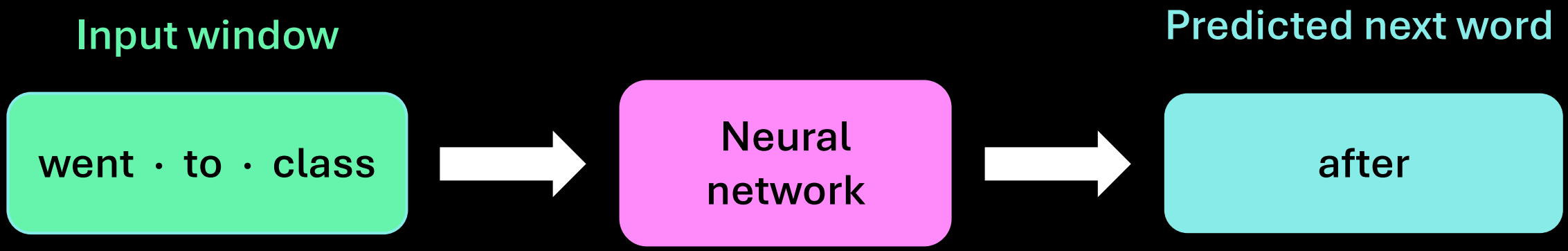
The Model Predicts the Next Word



She went to class.

The Same Network Predicts the Next Position

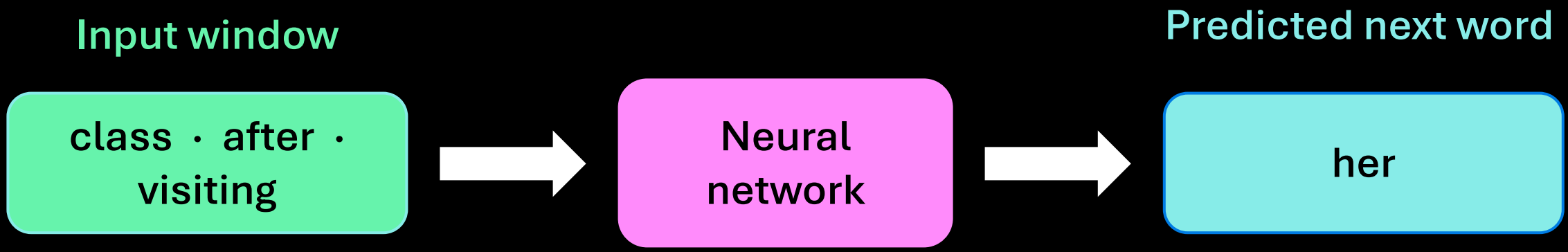
Move the fixed window one word forward.



Same neural network, new input window.

A Fixed Window Limits What the Model Can See

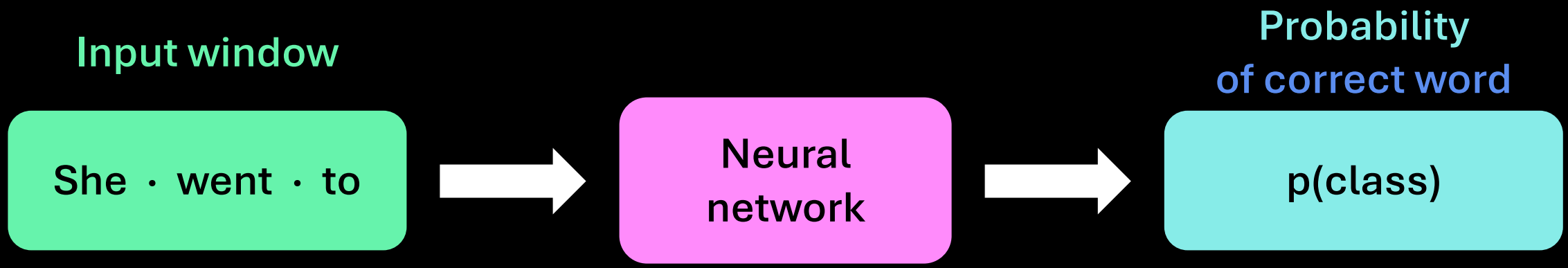
The model receives only the words in the current window.



The fixed window is all the model can see.

Next: how training pushes probability onto the correct word.

Training Rewards Probability on the Correct Next Word

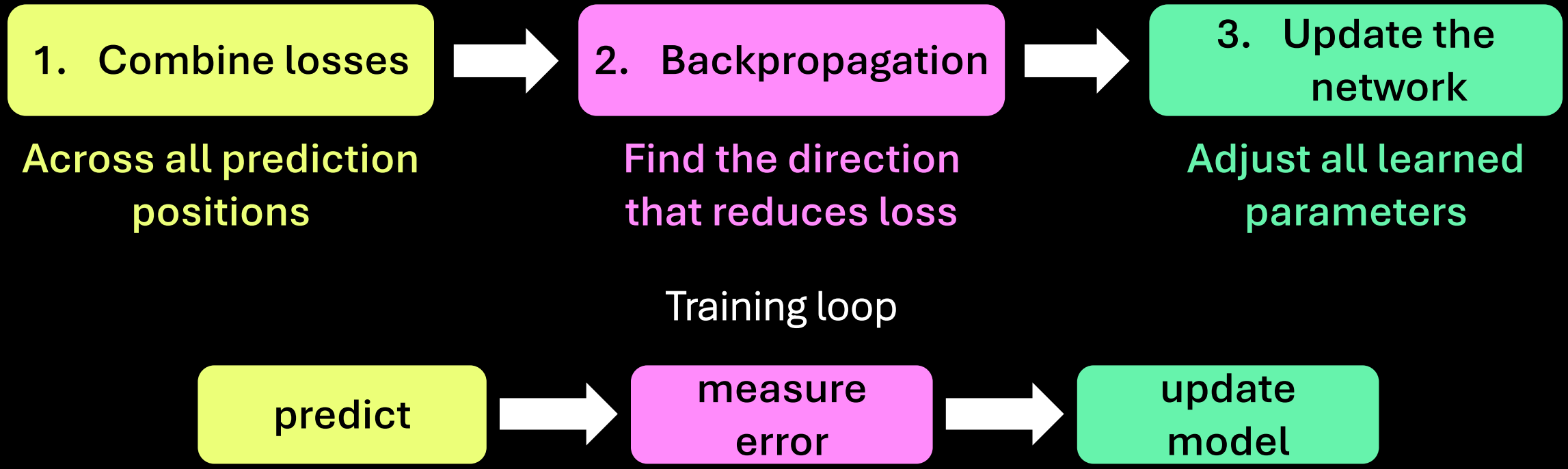


Correct next word: class

$$L = -\log p(\text{class})$$

Categorical cross-entropy for this example

Backpropagation Updates the Neural Network



Repeat over many examples.

Feed-Forward Language Models Have a Fixed Window

Strengths

Dense embeddings reduce sparsity

Learned parameters replace explicit n-gram counts

Limitation

The input window has a fixed size

A larger window adds many more weights and still cannot cover every dependency

We need a more flexible memory.



Outline

- Why Do We Need Neural Networks?
- Word Embeddings: Representing Words as Vectors
- A Feed-Forward Neural Language Model
- Training the Network: Loss and Backpropagation
- **Where We Go From Here**

Thank You!

See you in the next session!



COUNT EVERY 4-GRAM YOU EVER SAW



SHOVE 4 WORDS INTO A NEURAL NET



JUST REMEMBER THE WHOLE SENTENCE

Transformers

Pavlos Protopapas



Outline

- Motivation for Attention
- Attention Basics
- Building Blocks of Transformers, BERT and GPT
  - Multi-head Attention Block
  - Positional Encoding
- Bringing It All Together

Limitations

Language Model Wishlist?

- Need **long context**
- We want to capture the **context dependent** semantics of words

FCNN have fixed context.

Word2Vec gives us one embedding per word.

Limitations

Ambiguity in static embeddings:

Word2Vec gives us one embedding per word.

The **bank** is open on Fridays.

I went to the **bank** to take a walk by the river.

Limitations

Language Model Wishlist?

- Need long context
- We want to capture the **context dependent** semantics of words
- We want to respect the **sequential** nature of language
- We need an architecture that can be trained in **parallel** (non-Markovian property)

FCNN have fixed context.

Word2Vec gives us one embedding per word.

FCNN is not able to capture sequential information.

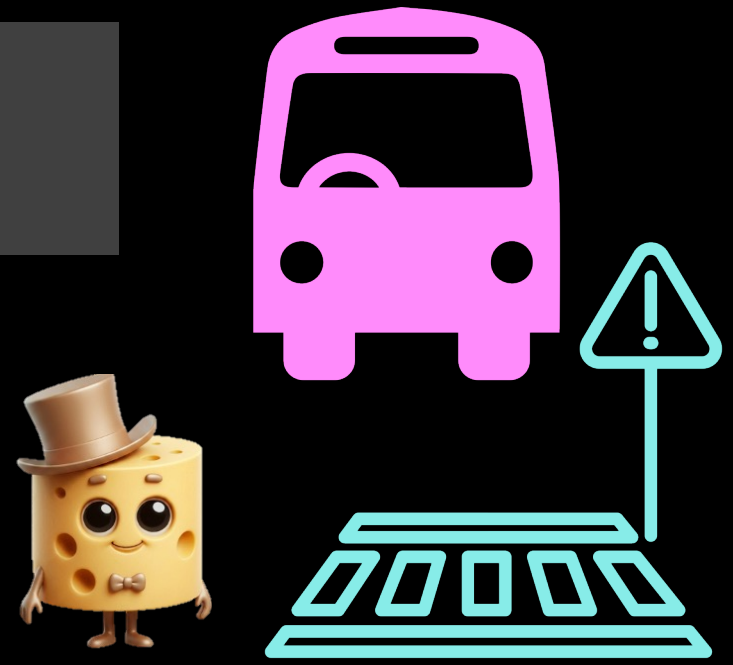
RNNs, GRUs and LSTM can not be trained in parallel as they rely on previous states.

Outline

- Motivation for Attention
- **Attention Basics**
- Building Blocks of Transformers, BERT and GPT
  - Multi-head Attention Block
  - Positional Encoding
- Bringing It All Together

Attention | Example

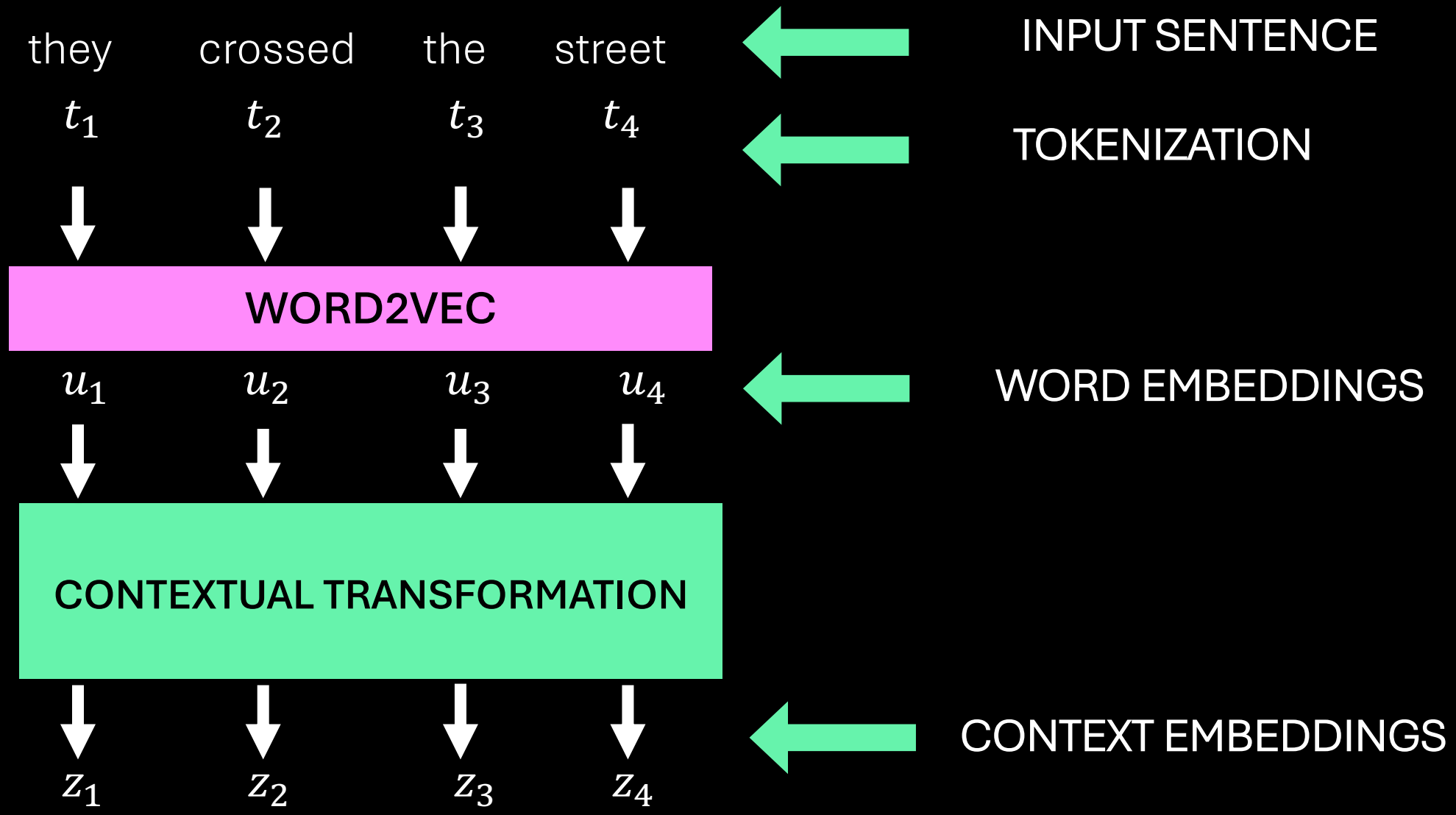
Chester was hit by a bus because **they** crossed the street.



How do we find the referent of the word '**they**' in the sentence?

No anthropomorphic cheeses were harmed in this lecture. This is a work of fiction. Any resemblance to actual people, places, events, or cheeses, living or otherwise, is purely coincidental.

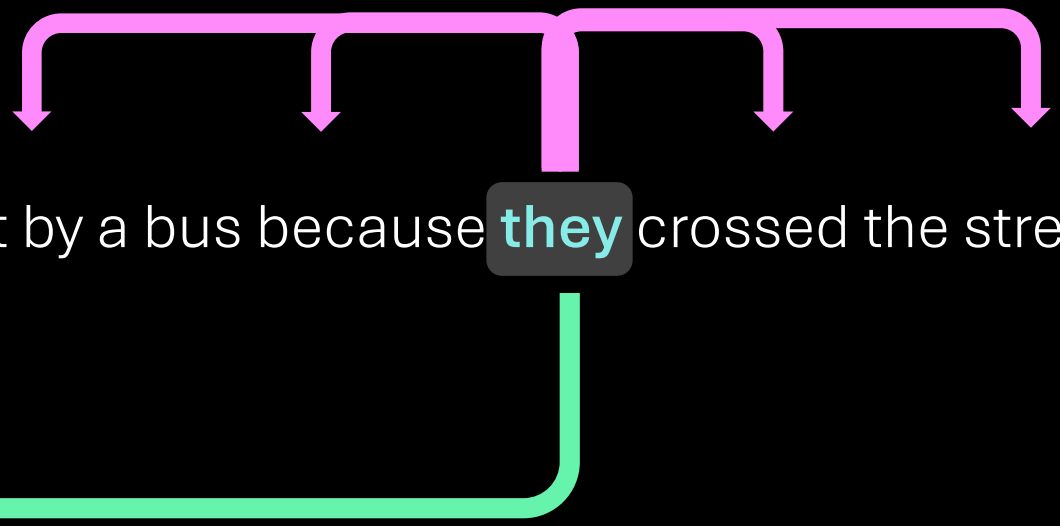
Contextual Embeddings



Attention | Example

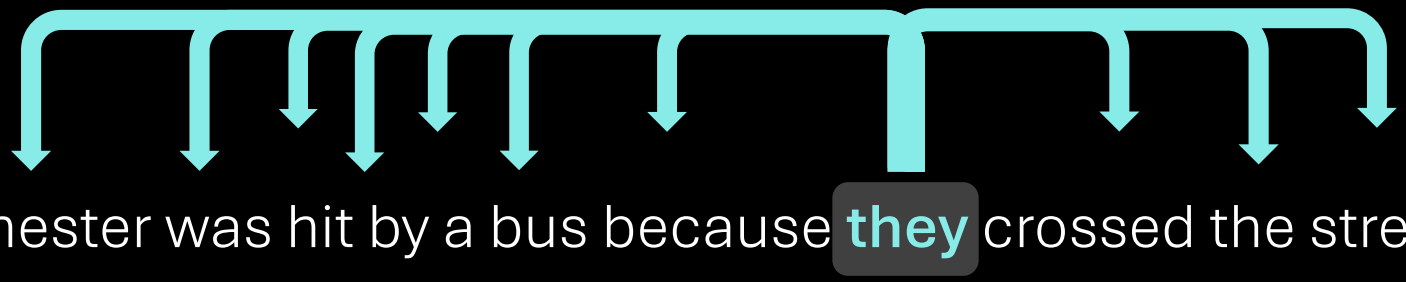


IDEA #1:
Distance relationship



🚫 This idea does not work because context can be **unevenly** spread out in a sentence.

Attention | Example



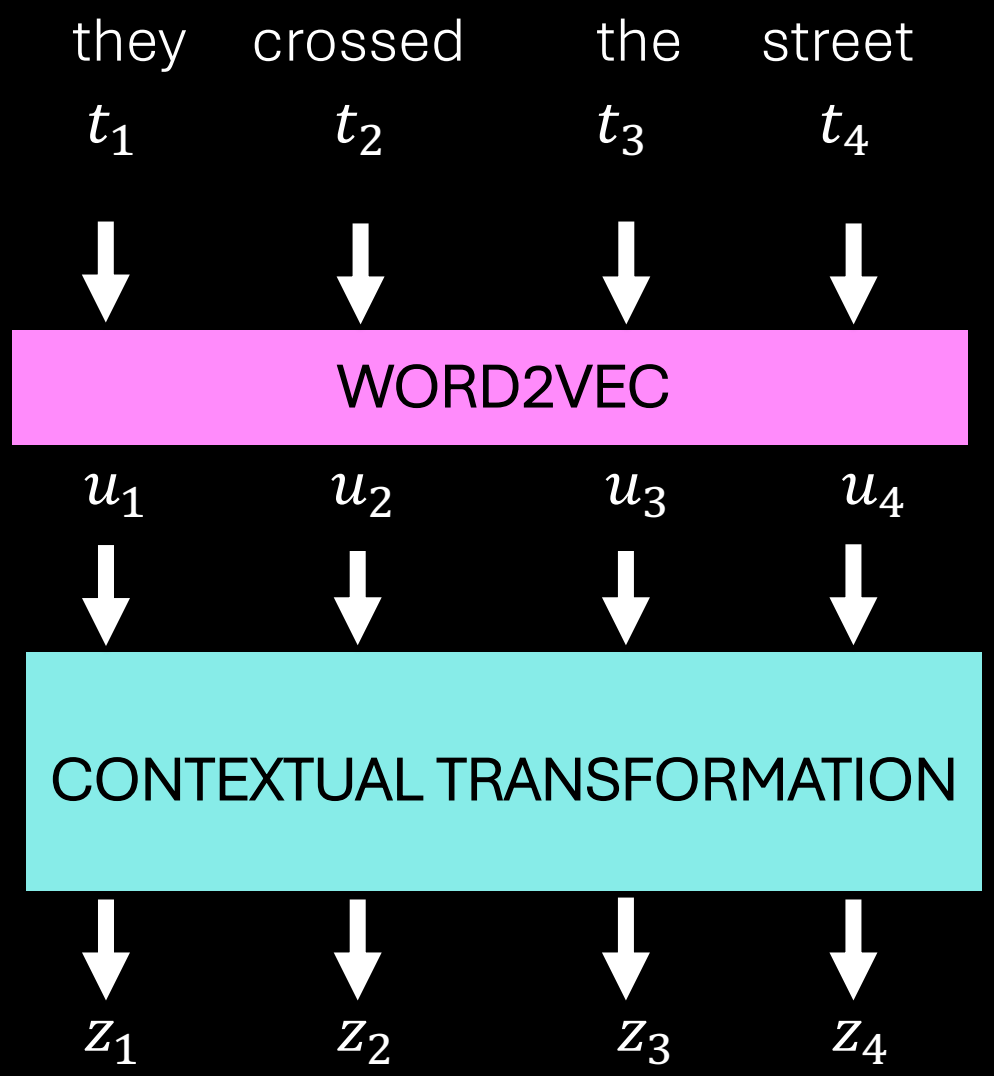
We would want the word '**they**' to be related to every other word in the sentence, irrespective of the distance.

One way to do this would be to make the contextual embedding of the word 'they' to be a **linear combination** of the other word embeddings.

Outline

- Motivation for Attention
- Attention Basics
- **Building Blocks of Transformers, BERT and GPT**
  - Multi-head Attention Block
  - Positional Encoding
- Bringing It All Together

Contextual Embeddings



Contextual embeddings, z, can be obtained by applying a linear transformation to the word embeddings, u.

So, we want:

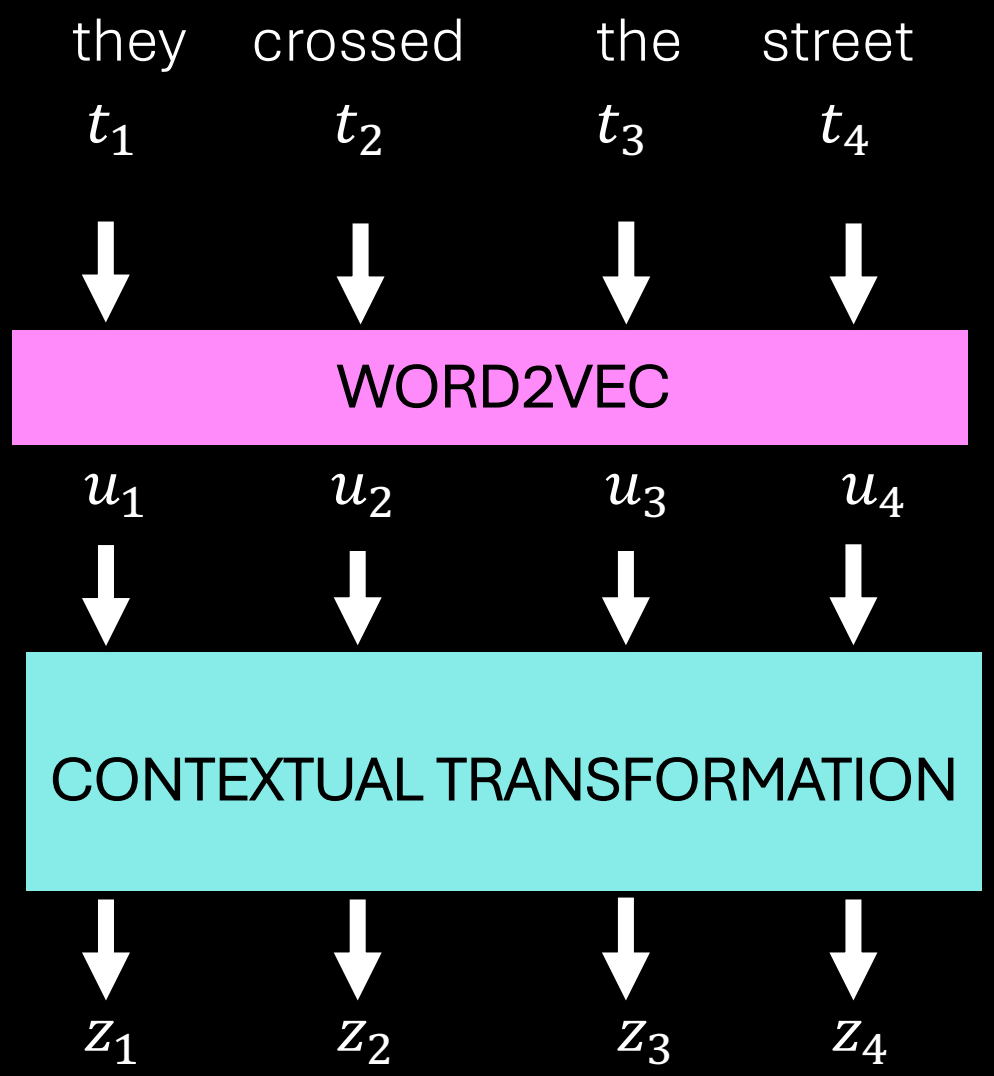
$$z_1 = \alpha_{11}u_1 + \alpha_{12}u_2 + \alpha_{13}u_3 + \alpha_{14}u_4$$

$$z_2 = \alpha_{21}u_1 + \alpha_{22}u_2 + \alpha_{23}u_3 + \alpha_{24}u_4$$

$$z_3 = \alpha_{31}u_1 + \alpha_{32}u_2 + \alpha_{33}u_3 + \alpha_{34}u_4$$

$$z_4 = \alpha_{41}u_1 + \alpha_{42}u_2 + \alpha_{43}u_3 + \alpha_{44}u_4$$

Contextual Embeddings



Contextual embeddings, z, can be obtained by applying a linear transformation to the word embeddings, u.

So, we want:

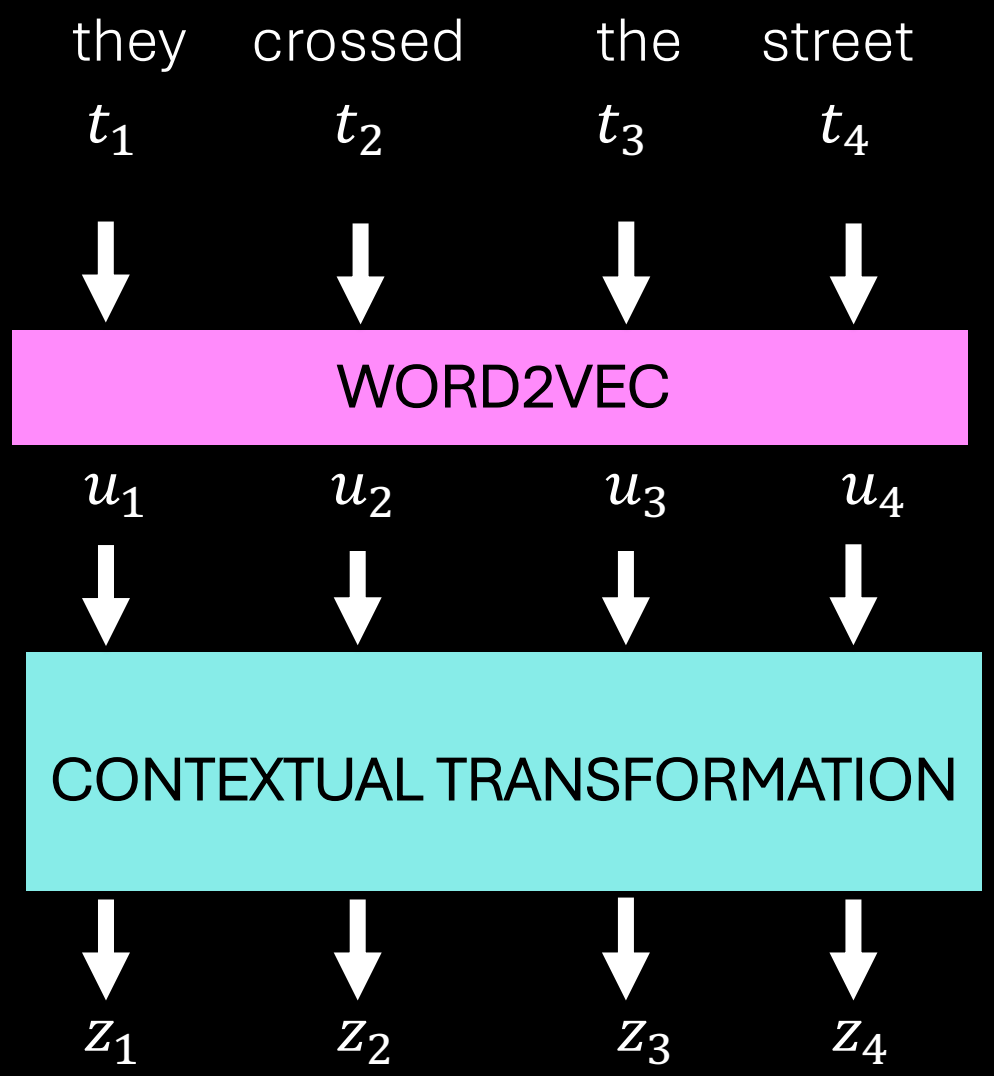
$$z_1 = \alpha_{11}u_1 + \alpha_{12}u_2 + \alpha_{13}u_3 + \alpha_{14}u_4$$

$$z_2 = \alpha_{21}u_1 + \alpha_{22}u_2 + \alpha_{23}u_3 + \alpha_{24}u_4$$

$$z_3 = \alpha_{31}u_1 + \alpha_{32}u_2 + \alpha_{33}u_3 + \alpha_{34}u_4$$

$$z_4 = \alpha_{41}u_1 + \alpha_{42}u_2 + \alpha_{43}u_3 + \alpha_{44}u_4$$

Contextual Embeddings



So, we want

$$z_1 = \alpha_{11}u_1 + \alpha_{12}u_2 + \alpha_{13}u_3 + \alpha_{14}u_4$$

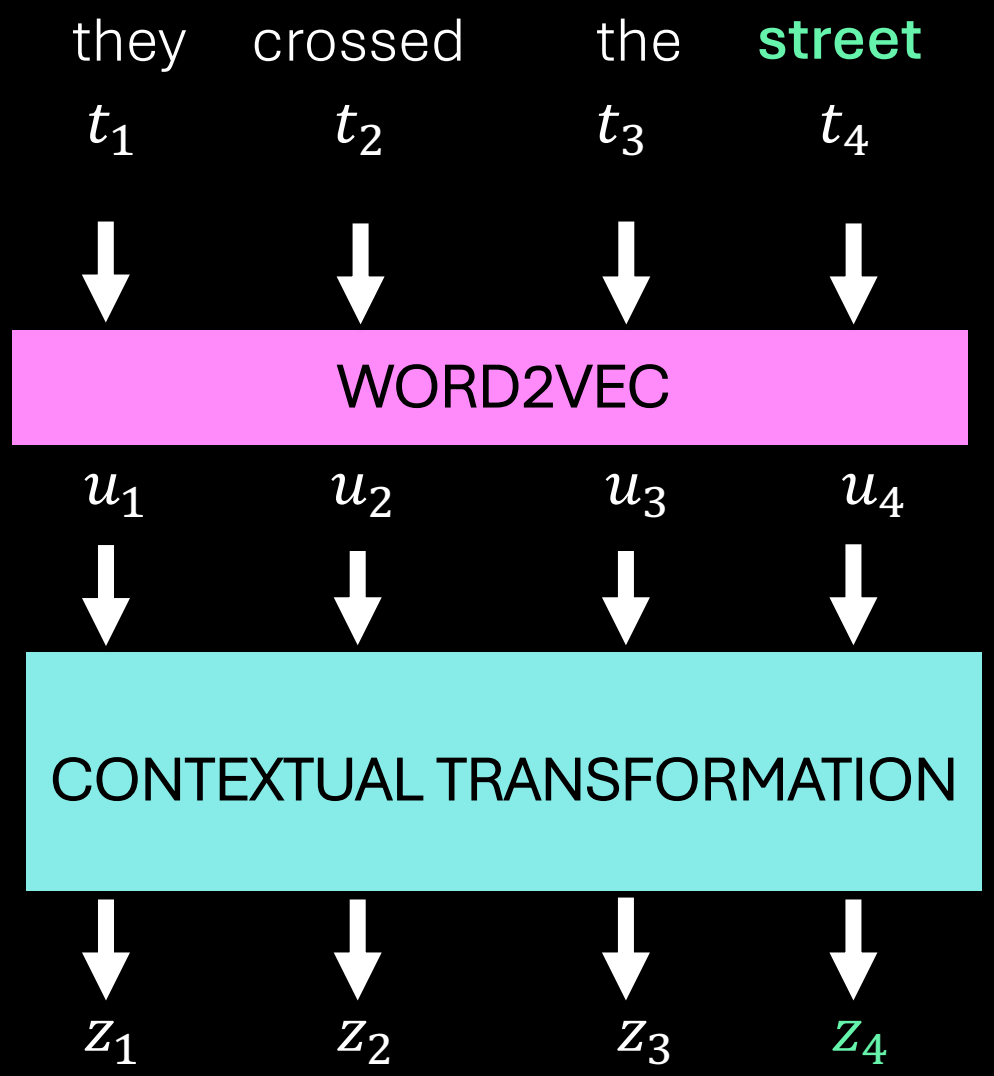
$$z_2 = \alpha_{21}u_1 + \alpha_{22}u_2 + \alpha_{23}u_3 + \alpha_{24}u_4$$

$$z_3 = \alpha_{31}u_1 + \alpha_{32}u_2 + \alpha_{33}u_3 + \alpha_{34}u_4$$

$$z_4 = \alpha_{41}u_1 + \alpha_{42}u_2 + \alpha_{43}u_3 + \alpha_{44}u_4$$

What are these coefficients?

Contextual Embeddings



So, we want

$$z_1 = \alpha_{11}u_1 + \alpha_{12}u_2 + \alpha_{13}u_3 + \alpha_{14}u_4$$

$$z_2 = \alpha_{21}u_1 + \alpha_{22}u_2 + \alpha_{23}u_3 + \alpha_{24}u_4$$

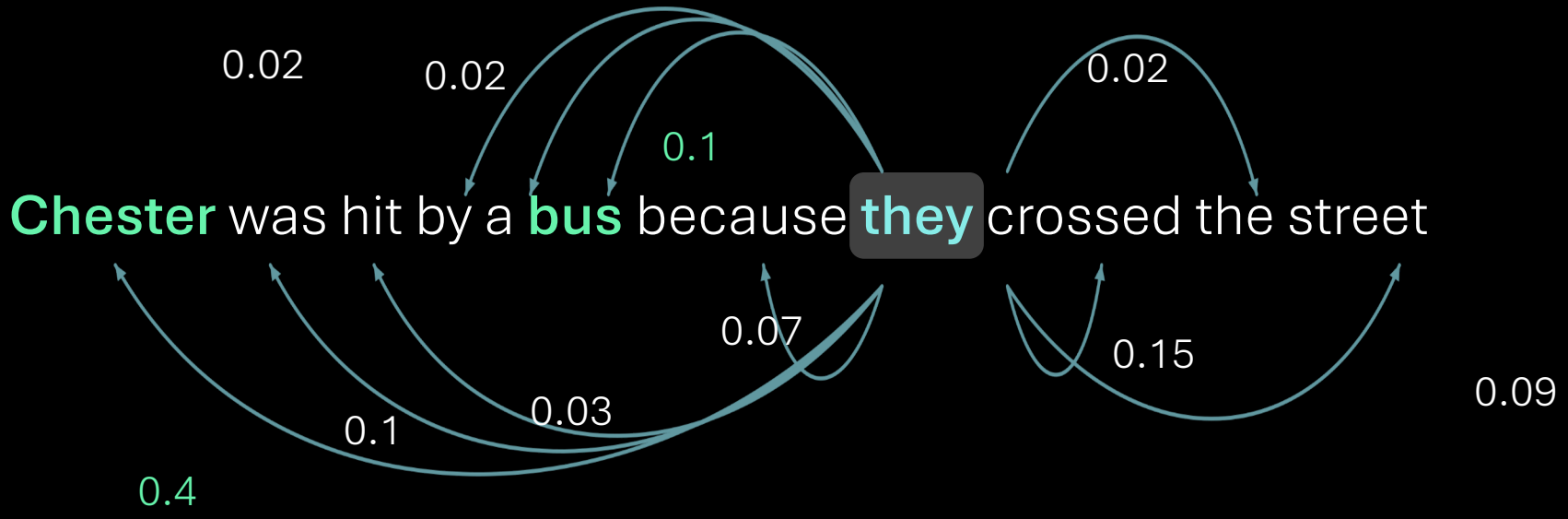
$$z_3 = \alpha_{31}u_1 + \alpha_{32}u_2 + \alpha_{33}u_3 + \alpha_{34}u_4$$

$$z_4 = \alpha_{41}u_1 + \alpha_{42}u_2 + \alpha_{43}u_3 + \alpha_{44}u_4$$

What are these coefficients?

Contextual Embeddings | Coefficients

The coefficients are weights that encapsulate the degree to which new contextual embeddings should incorporate the **influence** of the other tokens.

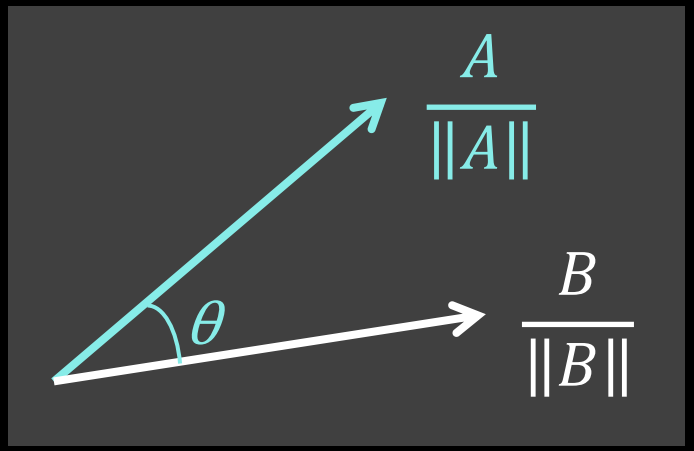


Contextual Embeddings | Coefficients



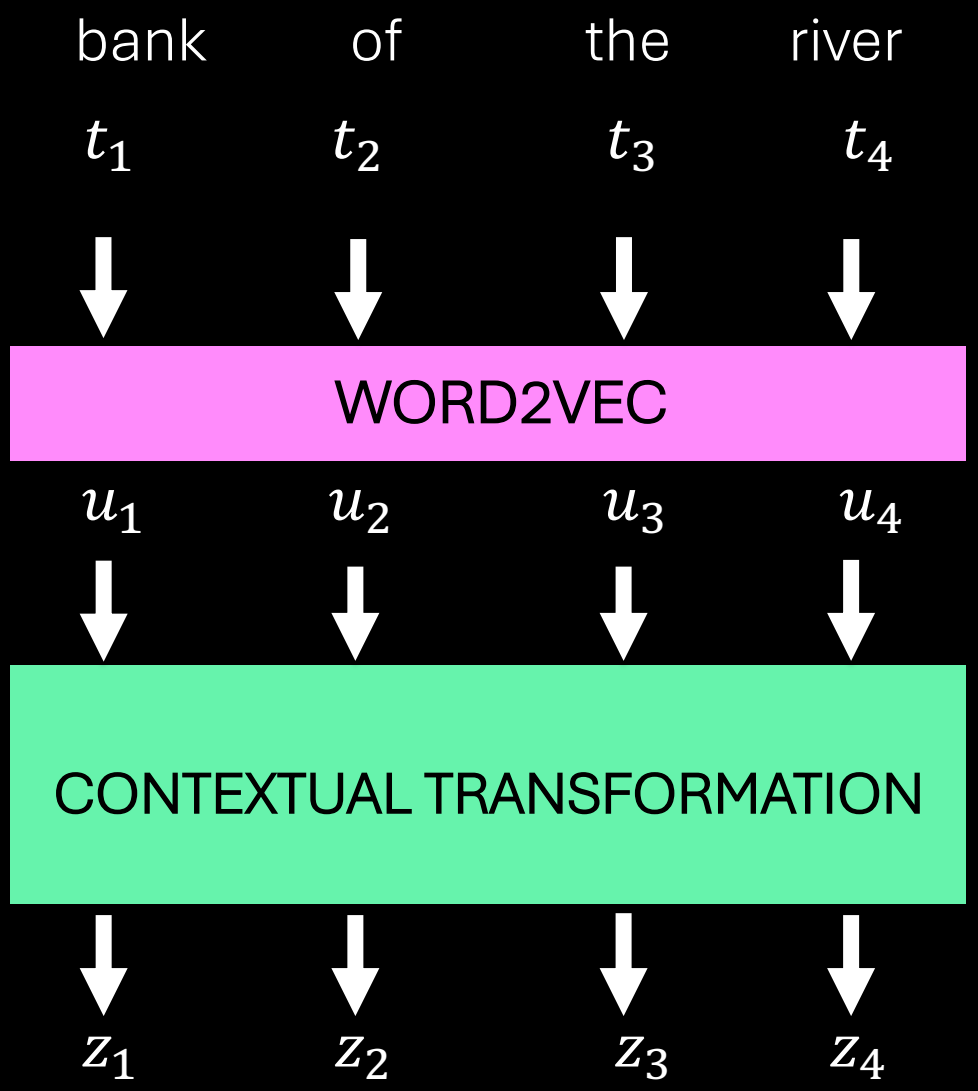
How do we calculate these coefficients?

We can use **cosine similarity** between the word embeddings as the coefficients.

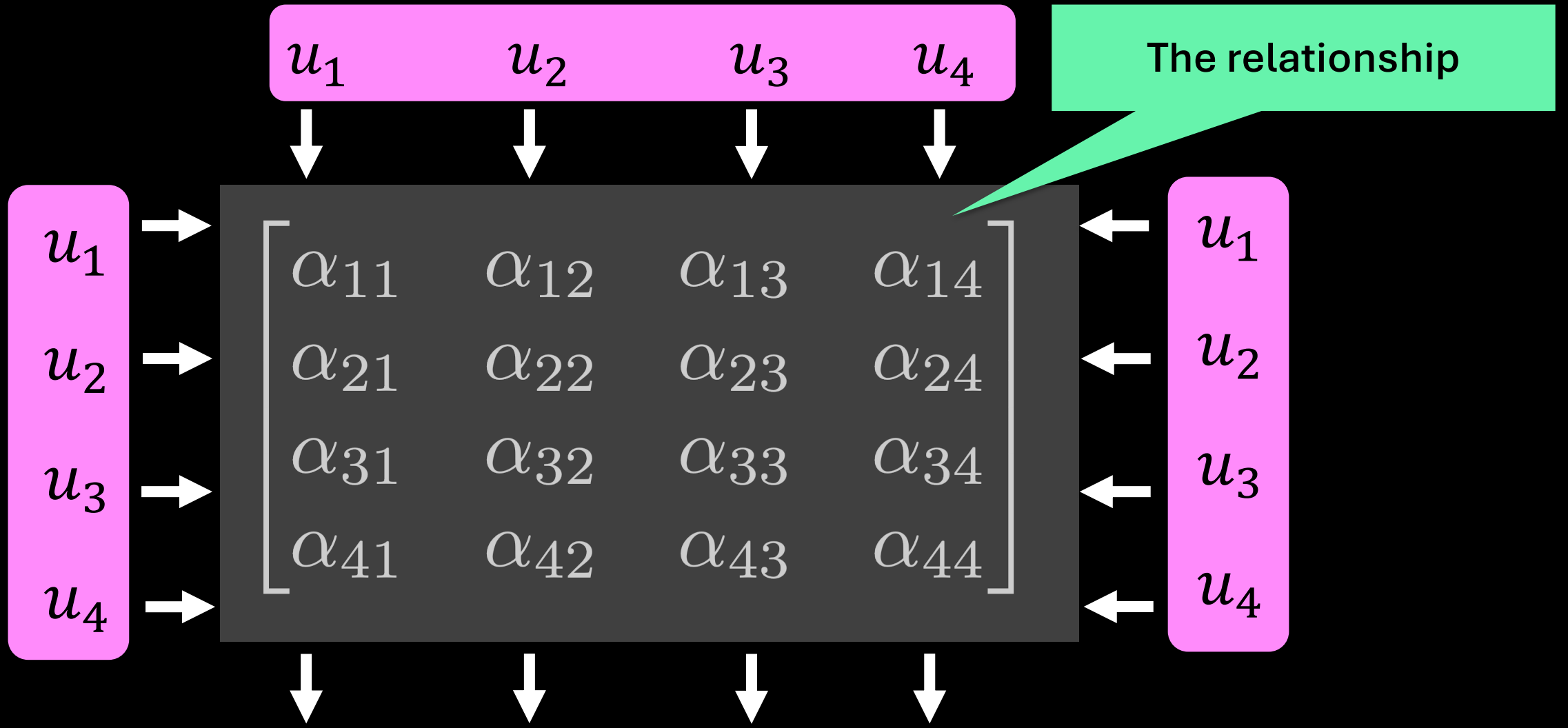


$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

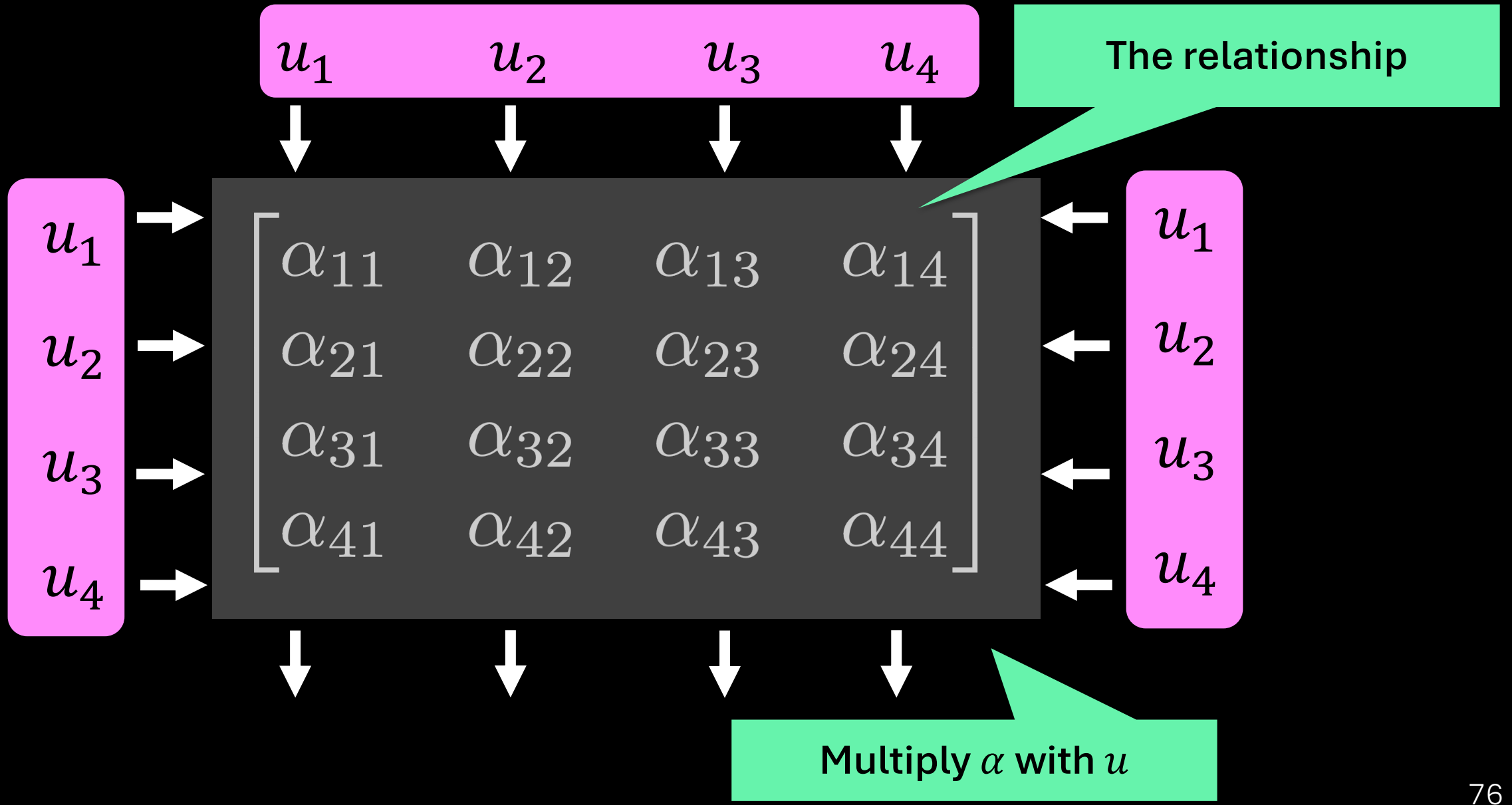
Attention | Example #2



Attention | Behind the Scenes



Attention | Behind the Scenes



Attention | Behind the Scenes

$u_1 \circ u_1 = s_{11}$

$u_1 \circ u_2 = s_{12}$

$u_1 \circ u_3 = s_{13}$

$u_1 \circ u_4 = s_{14}$

$\mathbf{softmax}(.)$



WEIGHTS NORMALIZATION

α_{11}

α_{12}

α_{13}

α_{14}

Attention | Behind the Scenes

$u_1 \circ u_1 = s_{11}$

$u_1 \circ u_2 = s_{12}$

$u_1 \circ u_3 = s_{13}$

$u_1 \circ u_4 = s_{14}$

softmax(.)

WEIGHTS NORMALIZATION

α_{11}

α_{12}

α_{13}

α_{14}

"Cosine similarity"

Attention | Behind the Scenes

$u_1 \circ u_1 = s_{11}$

$u_1 \circ u_2 = s_{12}$

$u_1 \circ u_3 = s_{13}$

$u_1 \circ u_4 = s_{14}$

softmax(.)

WEIGHTS NORMALIZATION

α_{11}

α_{12}

α_{13}

α_{14}

"Cosine similarity"

Attention | Behind the Scenes

Softmax is a function that scales numbers (logits, s_{ij}) into probabilities.

$$softmax(a_{ij}) = \frac{e^{s_{ij}}}{\sum_{j=1}^{k} e^{s_{ij}}}$$

$u_1 \circ u_1 = s_{11}$

$u_1 \circ u_2 = s_{12}$

$u_1 \circ u_3 = s_{13}$

$u_1 \circ u_4 = s_{14}$

softmax(.)

WEIGHTS NORMALIZATION

α_{11}

α_{12}

α_{13}

α_{14}

$$\Sigma \alpha_{1i} = 1$$

"Cosine similarity"

Attention | Behind the Scenes

$$z_1 = \alpha_{11}u_1 + \alpha_{12}u_2 + \alpha_{13}u_3 + \alpha_{14}u_4$$

New contextual
word embedding

Attention | Behind the Scenes

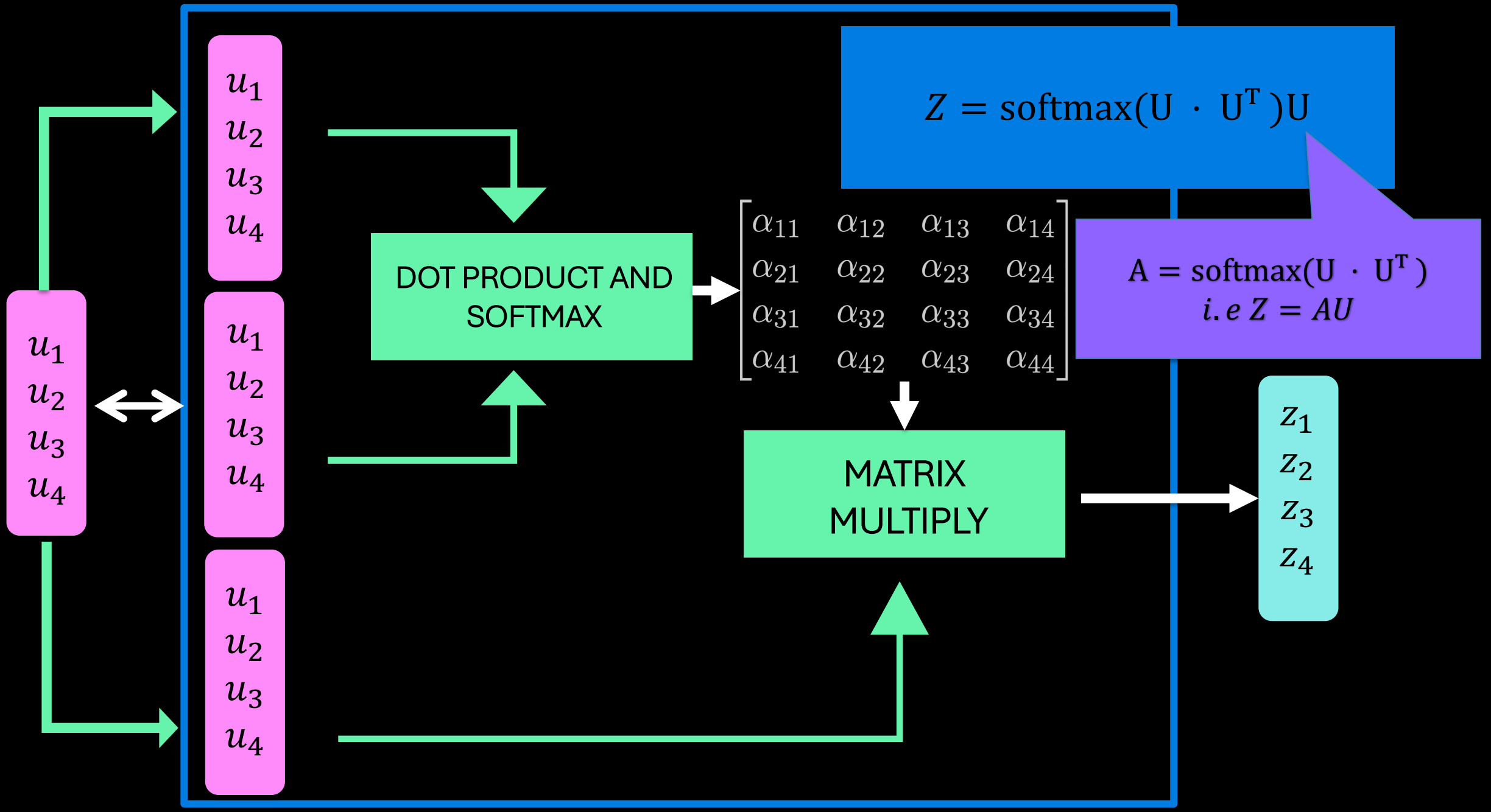
Similarly…

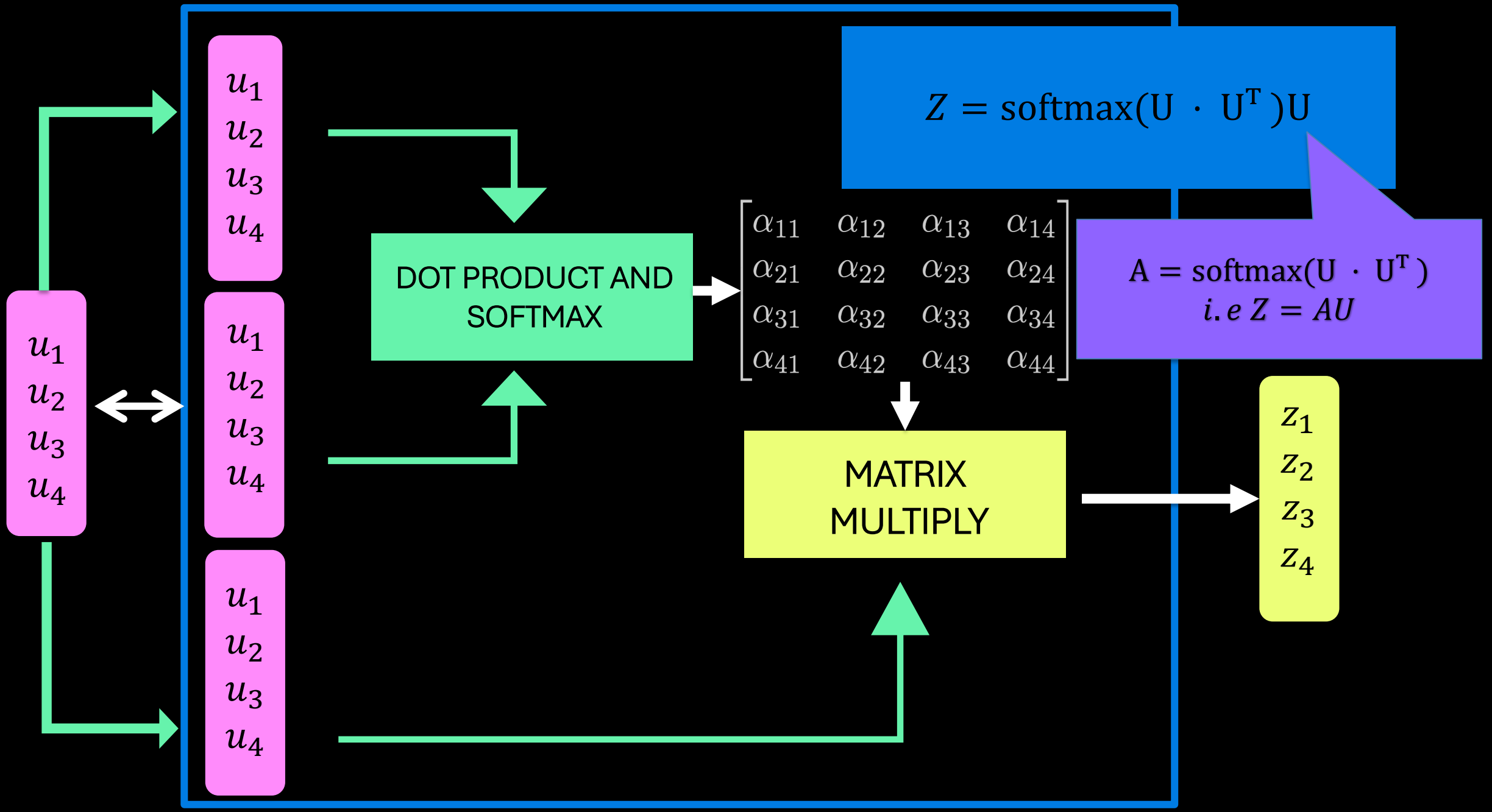
$$z_1 = \alpha_{11}u_1 + \alpha_{12}u_2 + \alpha_{13}u_3 + \alpha_{14}u_4$$

$$z_2 = \alpha_{21}u_1 + \alpha_{22}u_2 + \alpha_{23}u_3 + \alpha_{24}u_4$$

$$z_3 = \alpha_{31}u_1 + \alpha_{32}u_2 + \alpha_{33}u_3 + \alpha_{34}u_4$$

$$z_4 = \alpha_{41}u_1 + \alpha_{42}u_2 + \alpha_{43}u_3 + \alpha_{44}u_4$$


u_1 u_2 u_3 u_4
u_1 u_2 u_3 u_4
u_1 u_2 u_3 u_4
u_1 u_2 u_3 u_4
DOT PRODUCT AND SOFTMAX
α_{11} α_{12} α_{13} α_{14}
α_{21} α_{22} α_{23} α_{24}
α_{31} α_{32} α_{33} α_{34}
α_{41} α_{42} α_{43} α_{44}
$Z = \text{softmax}(\text{U} \cdot \text{U}^{\text{T}})\text{U}$
$\text{A} = \text{softmax}(\text{U} \cdot \text{U}^{\text{T}})$
$i.e\ Z = AU$
MATRIX MULTIPLY
z_1 z_2 z_3 z_4

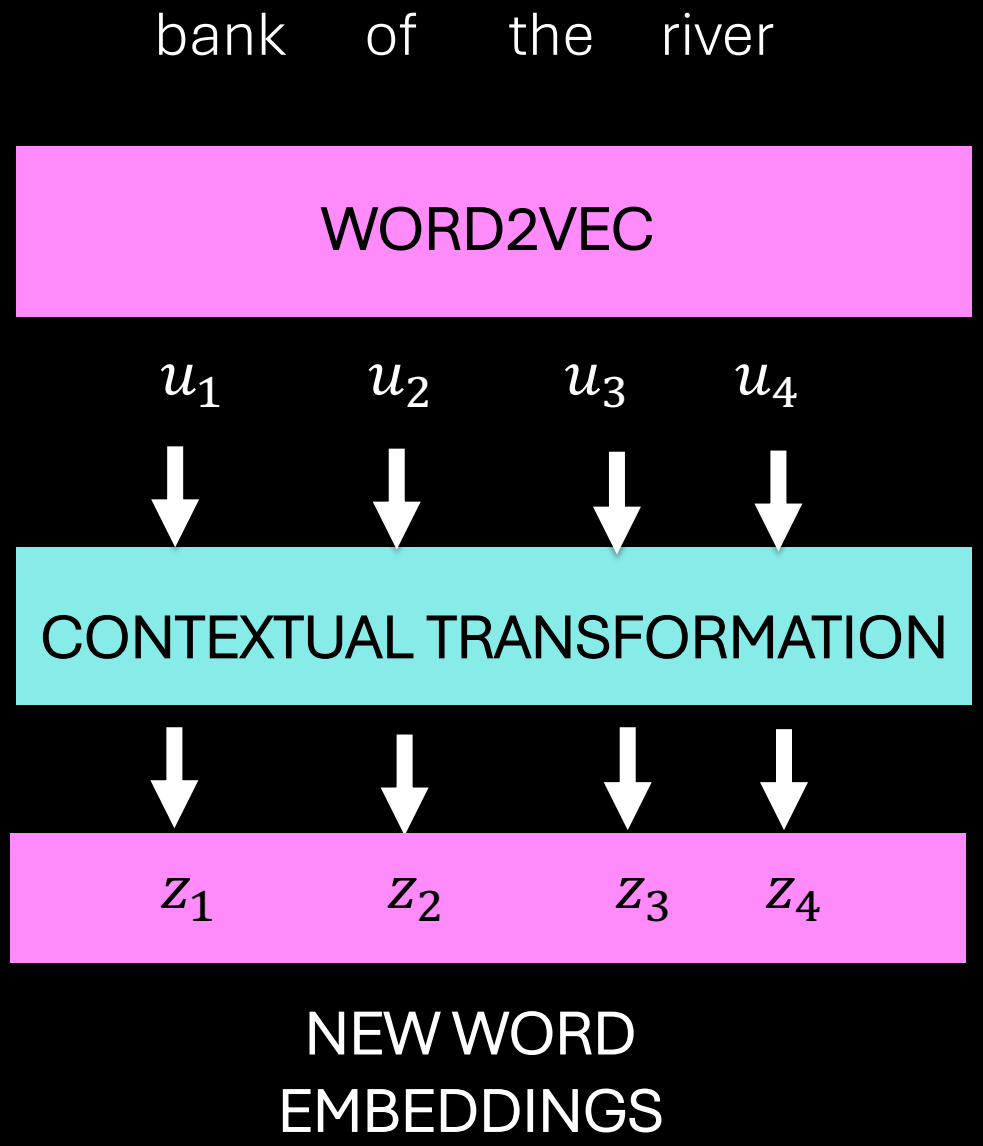

u_1 u_2 u_3 u_4
u_1 u_2 u_3 u_4
u_1 u_2 u_3 u_4
u_1 u_2 u_3 u_4
DOT PRODUCT AND SOFTMAX
α_{11} α_{12} α_{13} α_{14}
α_{21} α_{22} α_{23} α_{24}
α_{31} α_{32} α_{33} α_{34}
α_{41} α_{42} α_{43} α_{44}
$Z = \text{softmax}(\text{U} \cdot \text{U}^{\text{T}})\text{U}$
$\text{A} = \text{softmax}(\text{U} \cdot \text{U}^{\text{T}})$
$i.e\ Z = AU$
MATRIX MULTIPLY
z_1 z_2 z_3 z_4

Attention

ATTENTION ISSUES?

This process gives us new embeddings with some context. However, we still have the following issues:

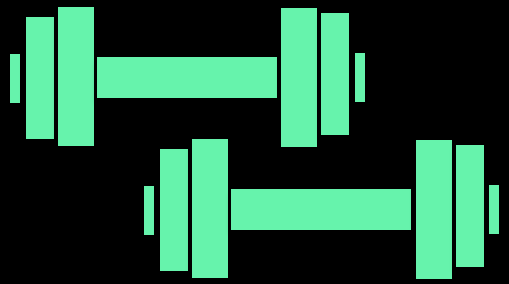
1. No weights are trained in the process.
2. The context captured is shallow — not enough representational power.
3. There is no positional information encoded.



Attention

Can we do even better?



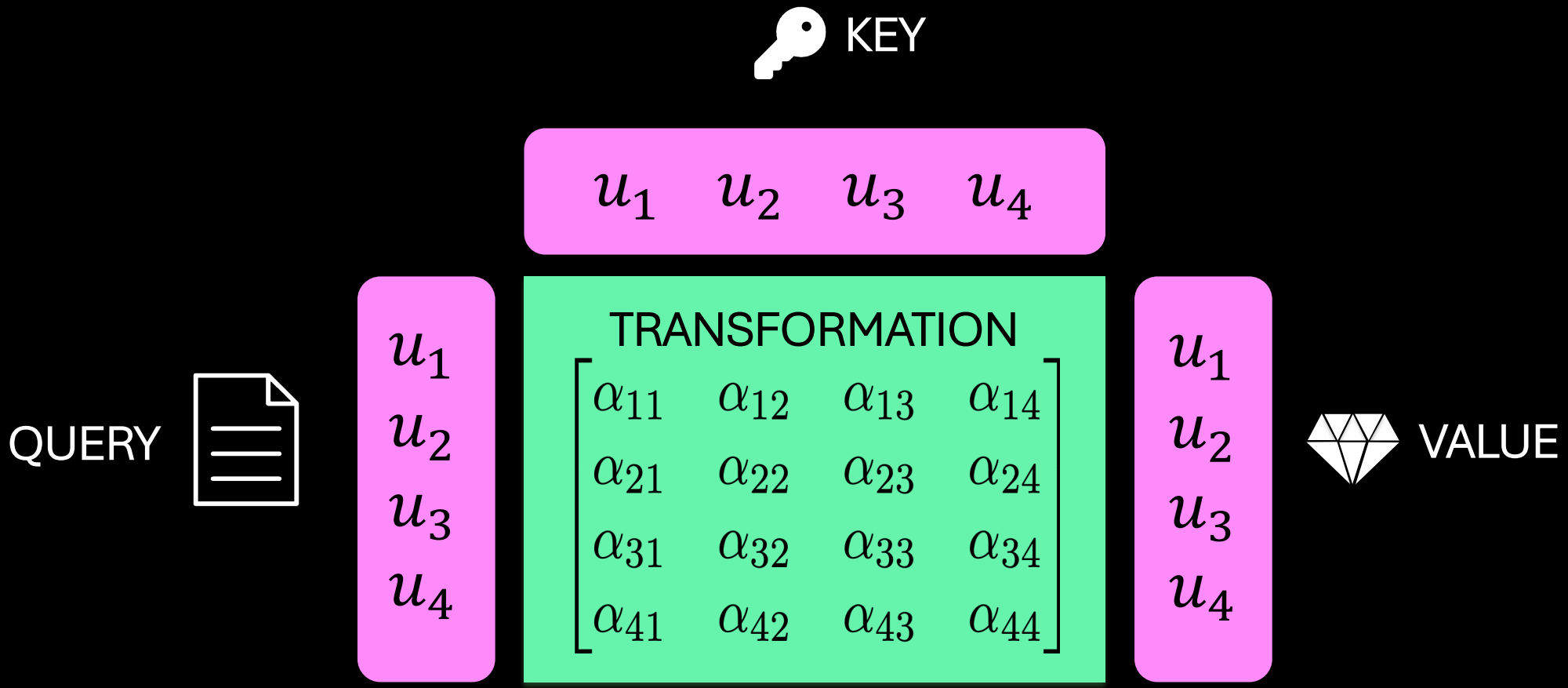


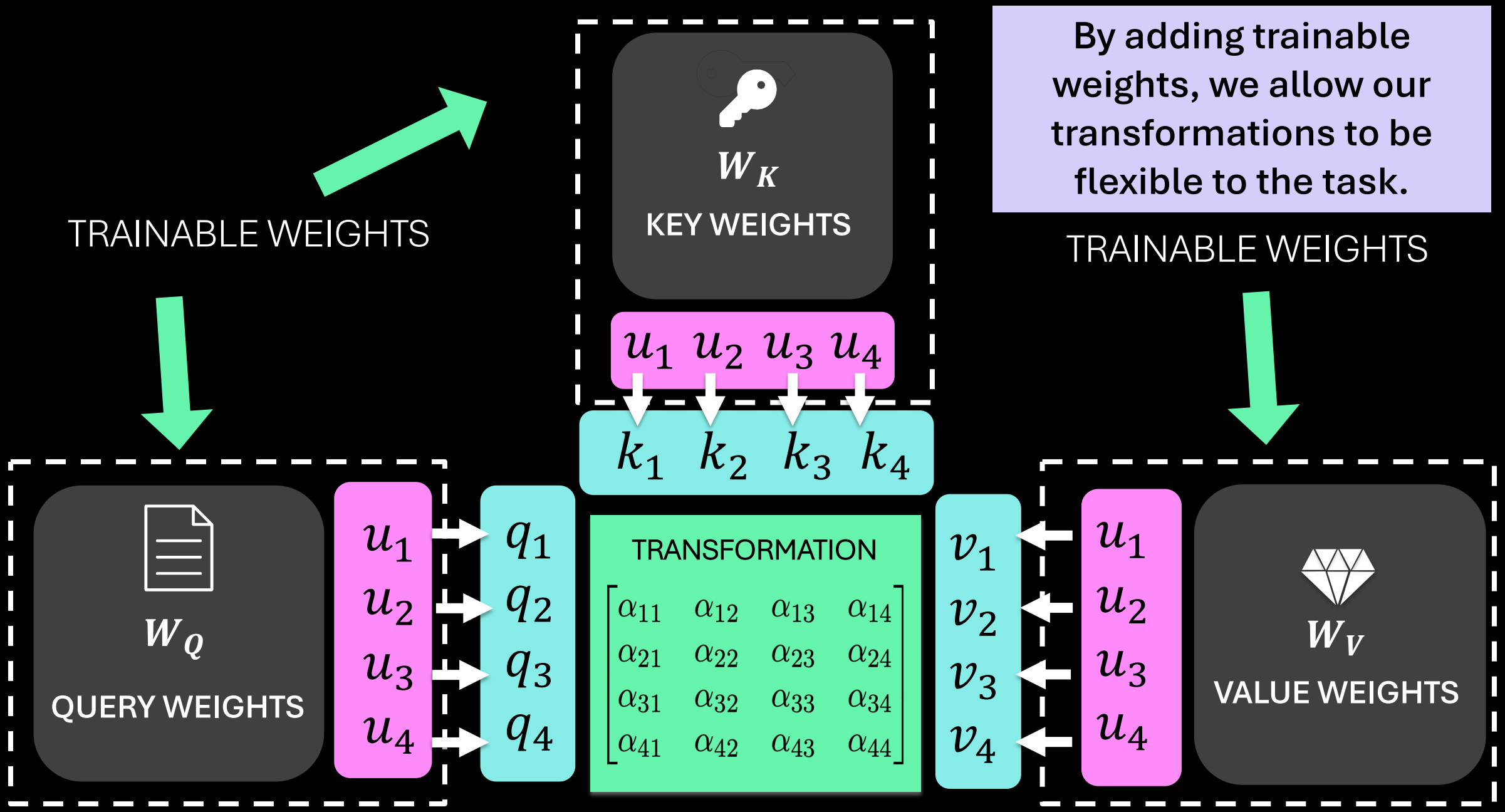
Yes! Let's add some weights and train.

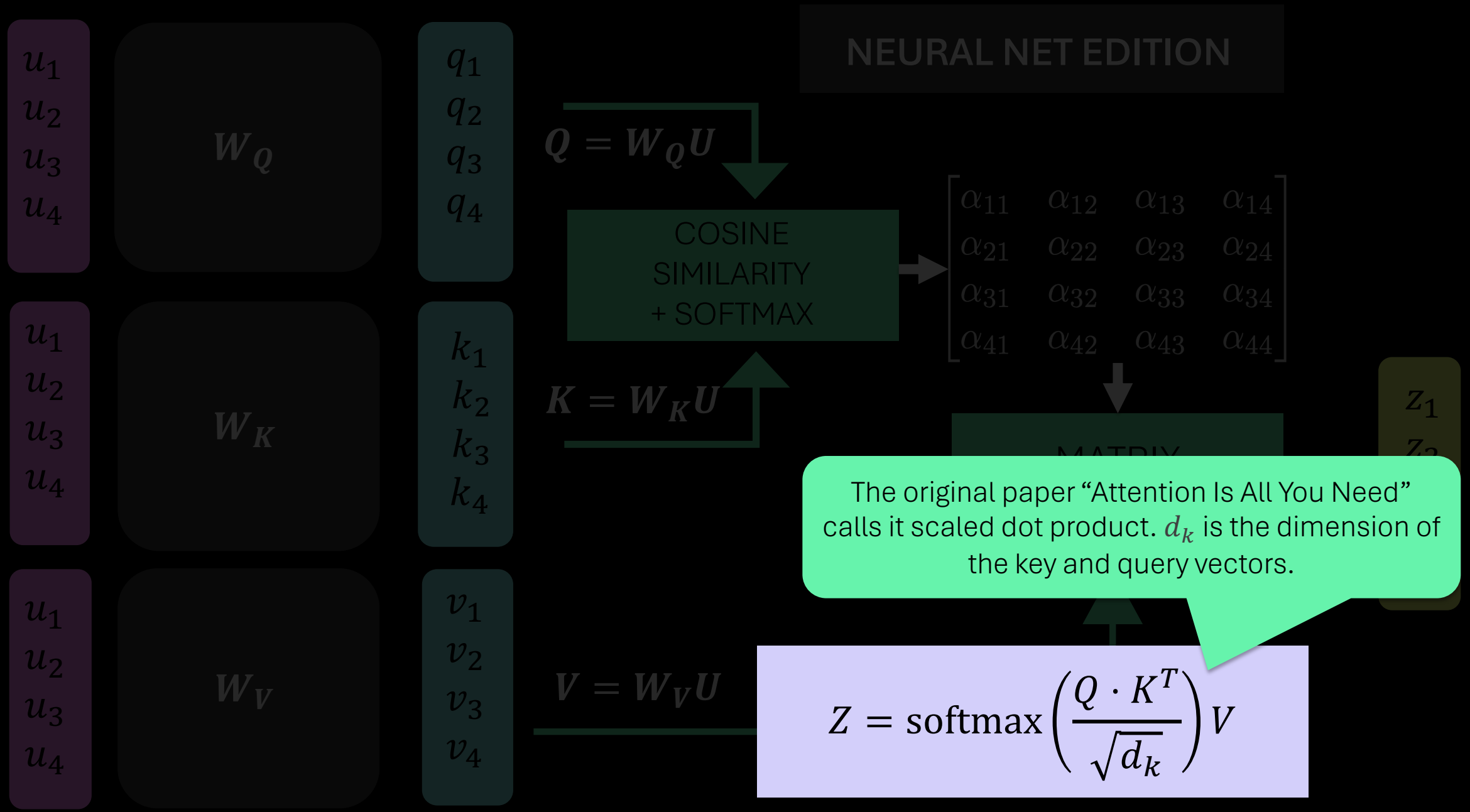


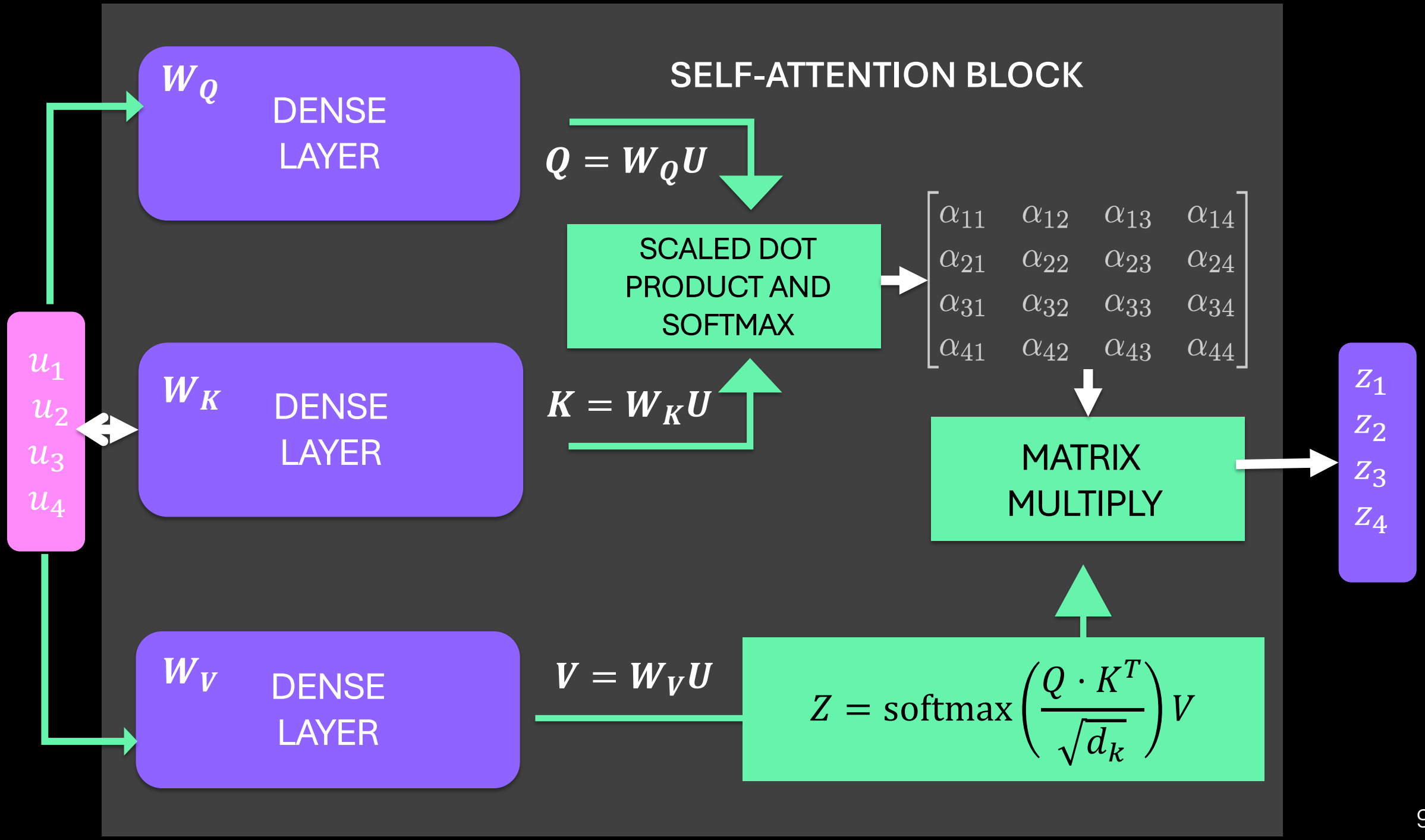
Attention | Database Analogy

For simplicity, we stick to our database analogy of QUERY, KEY & VALUE.




By adding trainable weights, we allow our transformations to be flexible to the task.
TRAINABLE WEIGHTS
TRAINABLE WEIGHTS
W_K
KEY WEIGHTS
u_1 u_2 u_3 u_4
k_1 k_2 k_3 k_4
W_Q
QUERY WEIGHTS
u_1 u_2 u_3 u_4
q_1 q_2 q_3 q_4
TRANSFORMATION
$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$
v_1 v_2 v_3 v_4
u_1 u_2 u_3 u_4
W_V
VALUE WEIGHTS


NEURAL NET EDITION
u_1 u_2 u_3 u_4
W_Q
q_1 q_2 q_3 q_4
$Q = W_Q U$
COSINE SIMILARITY + SOFTMAX
$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$
u_1 u_2 u_3 u_4
W_K
k_1 k_2 k_3 k_4
$K = W_K U$
z_1
The original paper “Attention Is All You Need” calls it scaled dot product. d_k is the dimension of the key and query vectors.
u_1 u_2 u_3 u_4
W_V
v_1 v_2 v_3 v_4
$V = W_V U$
$Z = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$


SELF-ATTENTION BLOCK
W_Q
DENSE LAYER
W_K
DENSE LAYER
W_V
DENSE LAYER
u_1
u_2
u_3
u_4
$Q = W_Q U$
$K = W_K U$
$V = W_V U$
SCALED DOT PRODUCT AND SOFTMAX
$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$
MATRIX MULTIPLY
z_1
z_2
z_3
z_4
$Z = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$

Attention

ATTENTION ISSUES?

1. ~~No weights are trained in the process.~~
2. Attention leads to **limited contextual mapping**.
3. There is no positional information encoded.



Attention | Do We Have Enough Attention?



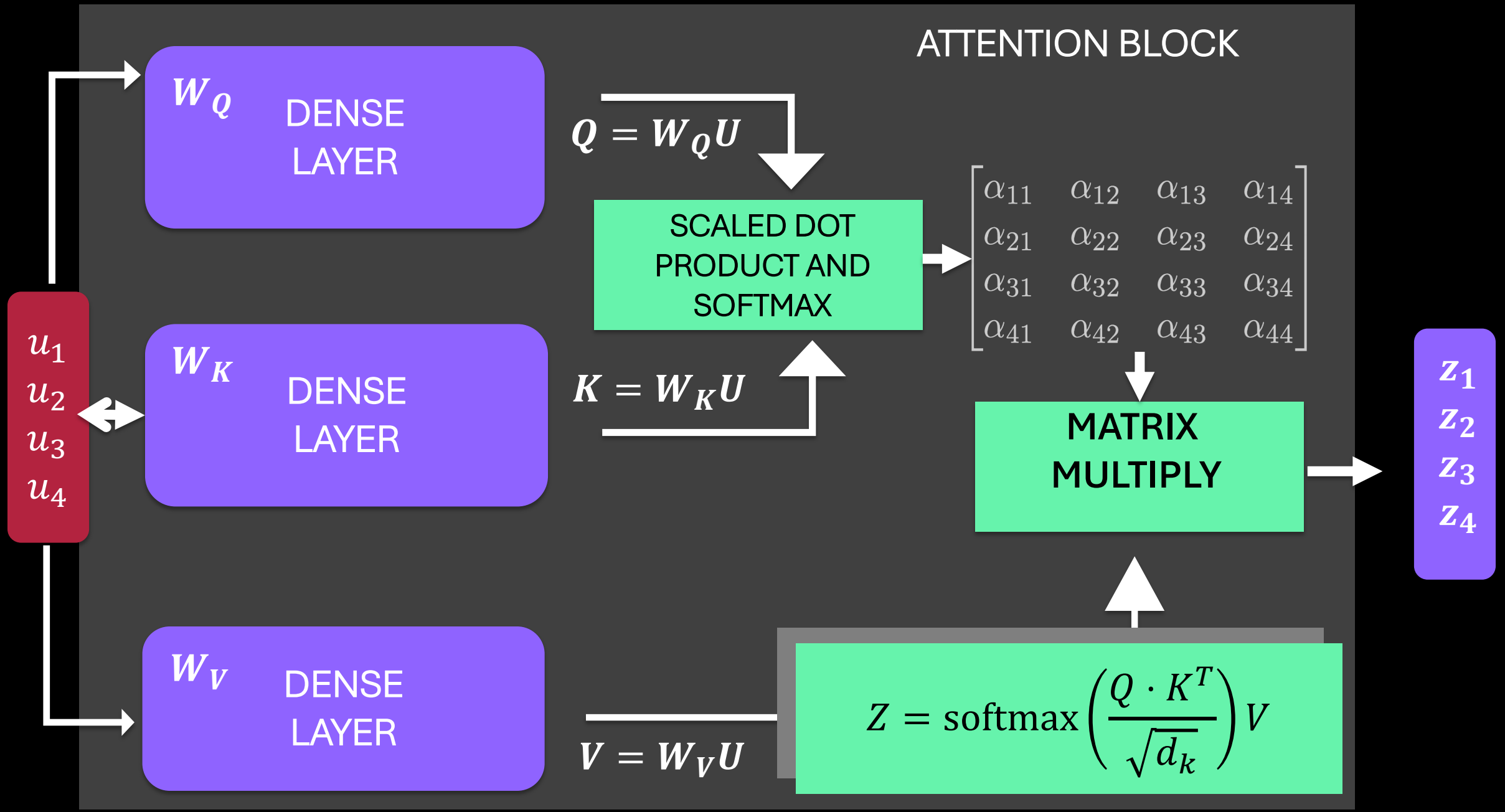
We need to have multiple attention mechanisms to look for different relations.

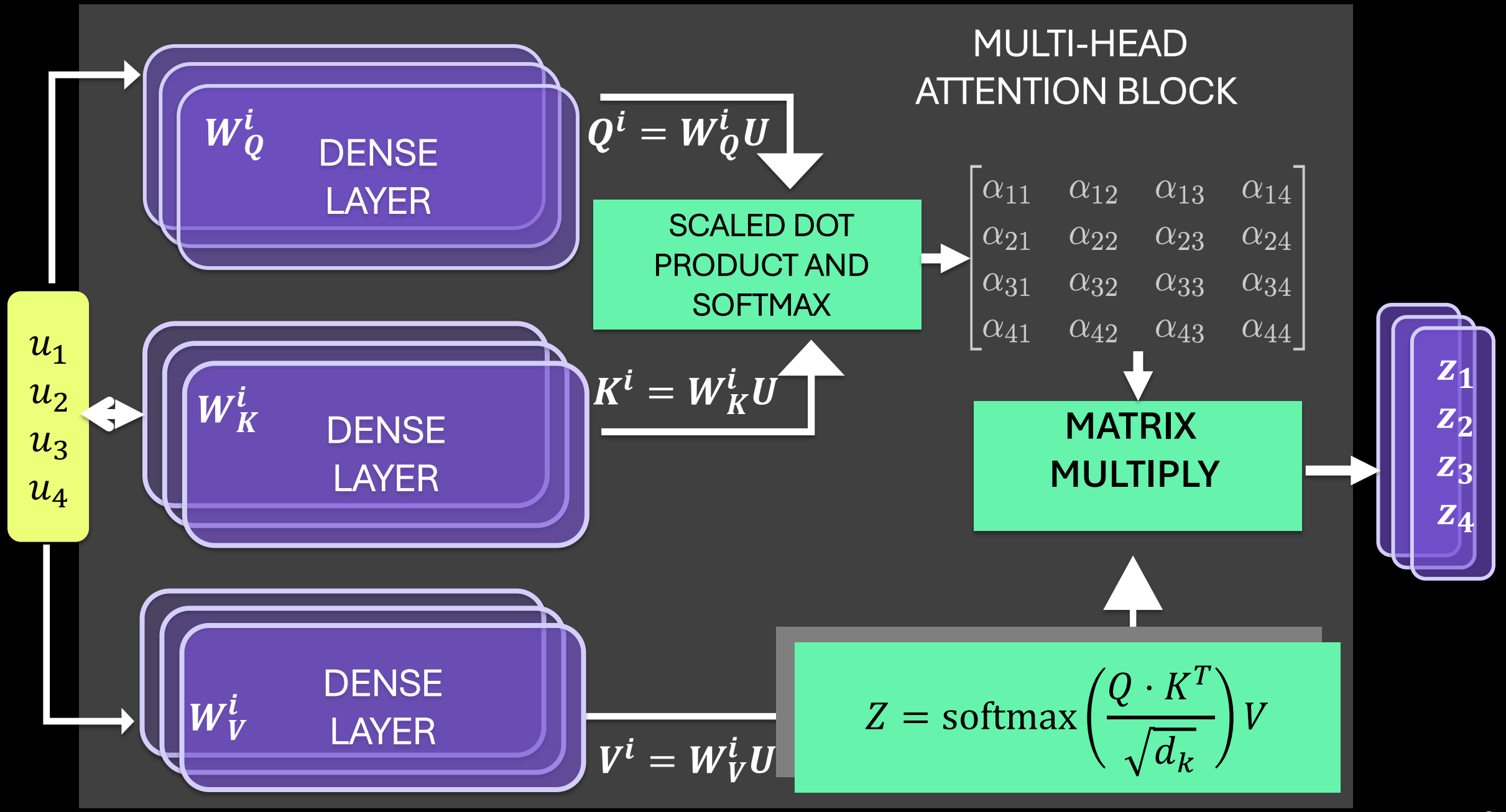
ATTENTION MECHANISM #1

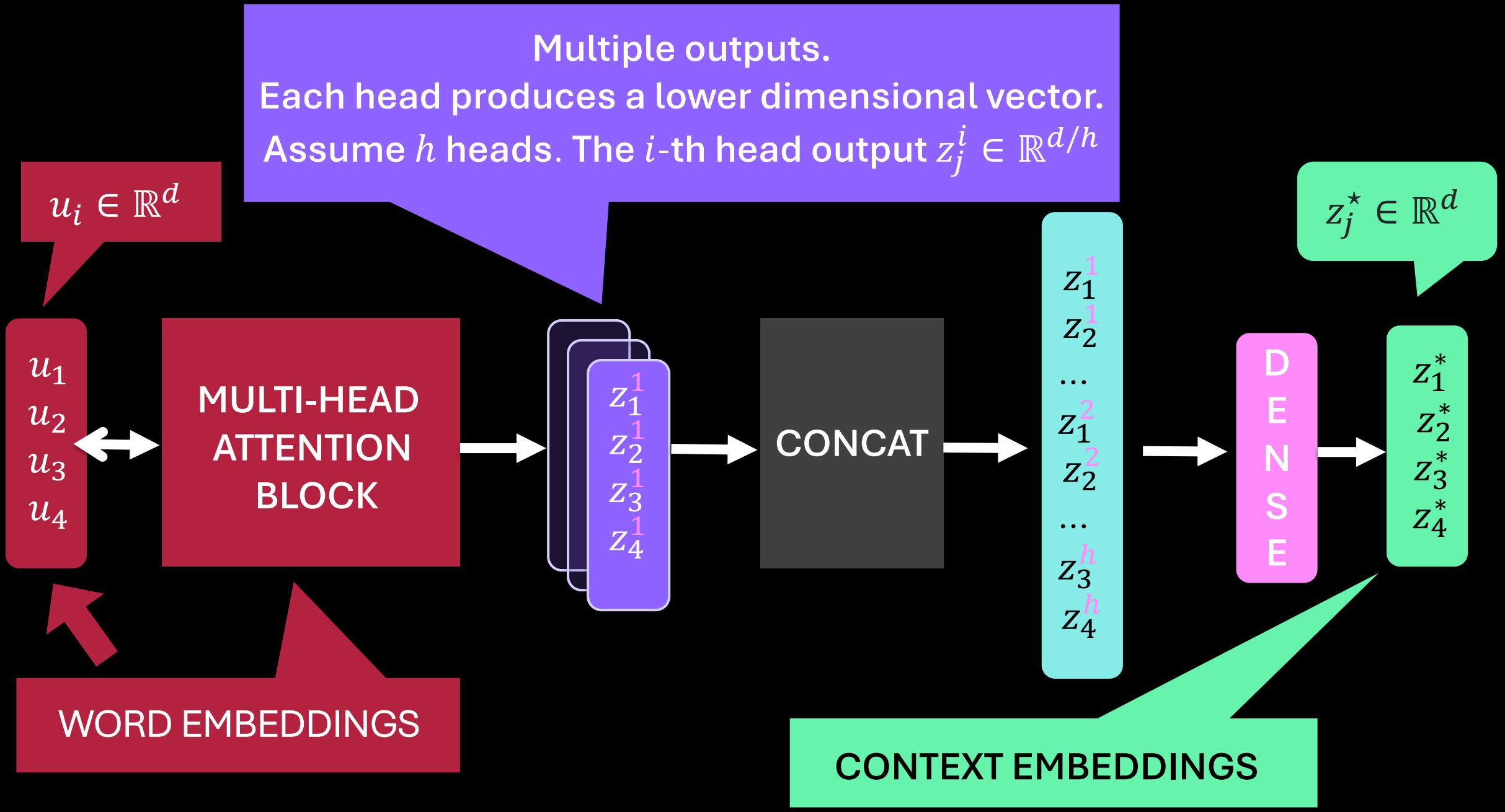
ATTENTION MECHANISM #2

Chester spoke to Pavlos about attention.

ATTENTION MECHANISM #3


ATTENTION BLOCK
W_Q
DENSE LAYER
W_K
DENSE LAYER
W_V
DENSE LAYER
u_1
u_2
u_3
u_4
$Q = W_Q U$
$K = W_K U$
$V = W_V U$
SCALED DOT PRODUCT AND SOFTMAX
$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$
MATRIX MULTIPLY
$Z = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$
z_1
z_2
z_3
z_4


MULTI-HEAD
ATTENTION BLOCK
W_Q^i
DENSE
LAYER
$Q^i = W_Q^i U$
SCALED DOT
PRODUCT AND
SOFTMAX
$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$
u_1
u_2
u_3
u_4
W_K^i
DENSE
LAYER
$K^i = W_K^i U$
MATRIX
MULTIPLY
z_1
z_2
z_3
z_4
W_V^i
DENSE
LAYER
$V^i = W_V^i U$
$Z = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$


Multiple outputs.
Each head produces a lower dimensional vector.
Assume h heads. The i-th head output $z_j^i \in \mathbb{R}^{d/h}$
$u_i \in \mathbb{R}^d$
u_1 u_2 u_3 u_4
MULTI-HEAD ATTENTION BLOCK
z_1^1 z_2^1 z_3^1 z_4^1
CONCAT
z_1^1 z_2^1 … z_1^2 z_2^2 … z_3^h z_4^h
DENSE
$z_j^\star \in \mathbb{R}^d$
z_1^* z_2^* z_3^* z_4^*
WORD EMBEDDINGS
CONTEXT EMBEDDINGS